

PROJECT REPORT
ON
“BREAST CANCER CLASSIFICATION USING MACHINE LEARNING”

Submitted in the partial fulfilment of the requirements for the award of Master’s Degree of

MASTER OF COMPUTER APPLICATION

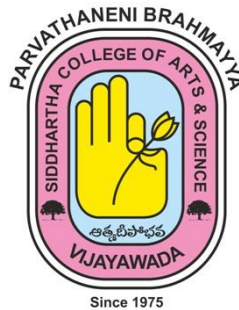
Submitted By

THOTA LOUKHYA [23MCA33]

Under the Guidance Of

Mrs. ROJA PRISCILLA MSc.[IS]

ASSITANT PROFESSOR



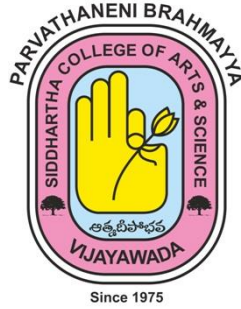
DEPARTMENT OF COMPUTER SCIENCE

P.B. SIDDHARTHA COLLEGE OF ARTS AND SCIENCE

Affiliated to Krishna University, Machilipatnam VIJAYAWADA-520 010

**PARVATHANENI BRAHMAYYA SIDDHARTHA COLLEGE OF ARTS
AND SCIENCE**

SIDDHARTHA NAGAR, VIJAYAWADA - 520 010



**DEPARTMENT OF COMPUTER SCIENCE
CERTIFICATE**

This is to certify that the project work entitled “**BREAST CANCER CLASSIFICATION USING MACHINE LEARNING**” is the Bonafide work of **THOTA LOUKHYA [23MCA33]** of Master of Computer Application submitted to Krishna University during the academic year 2024-2025.

PROJECT GUIDE
DEPARTMENT

HEAD OF THE

EXTERNAL EXAMINER

CERTIFICATE



CERTIFICATE

— OF COMPLETION —

THIS CERTIFICATE IS PRESENTED TO

Loukhya Thota

BREAST CANCER CLASSIFICATION

in recognition of the individual's commitment, skill, and contribution toward the
successful execution of the project from 09/04/2025 to 30/04/2025



RAHUL VARMA
COO, Fixity EDX

DECLARATION

I hereby declare that this project work entitled is “**BREAST CANCER CLASSIFICATION USING MACHINE LEARNING**” submitted by me to Krishna University during the academic year 2024-2025 impartial fulfilment of the requirements of Master of Computer Application.

I also declare that this project is the result of my own efforts, and it has not been submitted to any other institution or university for the award of any degree.

THOTA LOUKHYA [22MCA33]

ACKNOWLEDGEMENT

We are extremely thankful to our project guide B. Roja Priscilla Mam, M.Sc[IS], Assistant Professor, Department of Computer Science, P. B. Siddhartha College of Arts and Science, Krishna University, Machilipatnam for her timely cooperation and valuable suggestions throughout the project. I am indebted to her for the opportunity given to work under her guidance. I am extremely thankful to Dr. T. S. Ravi Kiran, Head of the Computer Science Department, and Dean Rajesh C. Jampala, P.B. Siddhartha College of Arts and Science, Vijayawada for providing a good environment and better infrastructure facilities.

My sincere thanks to all the teaching and non-teaching staff of Department of Computer Science for their support throughout my project work. Finally, I am very much indebted to my family for their moral support and encouragement to achieve higher goals.

CONTENTS

SNO	TITLE	PAGE NUMBERS
	ABSTRACT	1
1	INTRODUCTION 1.1 PROBLEM STATEMENT 1.2 MOTIVATION 1.3 OBJECTIVE 1.4 RESEARCH	2-6
2	SYSTEM REQUIREMENTS 2.1 HARDWARE REQUIREMENTS 2.2 SOFTWARE REQUIREMENTS	7-8
3	SYSTEM ANALYSIS 3.1 PYTHON 3.2 MACHINE LEARNING 3.3 MODULES	9-34
4	SYSTEM DESIGN 4.1 SYSTEM ARCHITECTURE 4.2 ACTIVITY DIAGRAM 4.3 PRELIMINARY INVESTIGATION	35-42
5	SOURCE CODE	43-59
6	CONCLUSION	60-61
7	FUTURE SCOPE	62-63
8	BIBLIOGRAPHY	64-65
9	RESEARCH REPORT	66-81

ABSTRACT

The solution approach involves developing a machine learning model capable of classifying breast tumors as either benign or malignant based on clinical features extracted from biopsy samples. By leveraging the well-known Wisconsin Breast cancer is one of the leading causes of death among women worldwide. Early detection and diagnosis are crucial to increasing survival rates. Manual diagnosis methods can be time-consuming and prone to error. This project aims to build a machine learning model that classifies tumors as benign or malignant using features derived from digitized images of a fine needle aspirate (FNA) of a breast mass. The model assists in reliable and efficient medical diagnosis. This project addresses the challenge of automating breast cancer classification to assist medical professionals in making quicker and more accurate diagnosis. The core methodology includes data preprocessing, splitting the dataset into training and testing sets, and training a logistic regression model. Breast cancer remains one of the leading causes of death among women worldwide, making early detection crucial for improving survival rates. This project addresses the challenge of automating breast cancer classification to assist medical professionals in making quicker and more accurate diagnosis. Breast Cancer dataset, the project utilizes structured data to build a supervised learning pipeline focused on binary classification. The core methodology includes data preprocessing, splitting the dataset into training and testing sets, and training a logistic regression model. Key technologies employed in this project are Python, pandas, NumPy, matplotlib for visualization, seaborn for data exploration, and scikit-learn for model development and evaluation. Accuracy is used as the primary metric to assess model performance. The final model demonstrates high classification accuracy, indicating its reliability in distinguishing between malignant and benign tumors. The use of a simple yet powerful logistic regression model offers interpretability and effectiveness, making it a practical tool for initial screenings. This work provides a foundation for further improvements and integration into healthcare decision-support systems.

INTRODUCTION

1.INTRODUCTION

1.1 Problem statement

Breast cancer is one of the leading causes of death among women worldwide. Early detection and diagnosis are crucial to increasing survival rates. Manual diagnosis methods can be time-consuming and prone to error. This project aims to build a machine learning model that classifies tumors as benign or malignant using features derived from digitized images of a fine needle aspirate (FNA) of a breast mass. The model assists in reliable and efficient medical diagnosis.

This project proposes a machine learning-based classification system using logistic regression, aimed at identifying whether a tumor is benign or malignant. By automating this classification, the system helps address inefficiencies in manual diagnostics and provides a scalable tool for mass screening.

1.2 Motivation:

Breast cancer continues to be one of the leading causes of death among women globally. Early detection is vital for improving survival rates, yet traditional diagnostic methods such as manual analysis of biopsy results are often time-consuming, subjective, and heavily dependent on expert availability.

The motivation for this project stems from the need to develop a reliable, efficient, and interpretable diagnostic tool to assist medical practitioners in the early detection and classification of breast tumours. By leveraging machine learning — specifically a logistic regression model — the project aims to automate the classification of tumours as benign or malignant based on structured clinical data from the Wisconsin Breast Cancer dataset.

The use of machine learning not only addresses inefficiencies in manual diagnosis but also introduces scalability, consistency, and speed to the diagnostic process. Furthermore, logistic regression, ensures that the model's predictions are interpretable by healthcare providers, thereby building trust and facilitating clinical adoption.

Ultimately, this project is motivated by the goal of enhancing diagnostic accuracy, improving patient outcomes, and contributing towards democratizing access to quality healthcare diagnostics through technology.

1.3 Objective:

The main objective of this project is to develop a reliable and interpretable machine learning model that can accurately classify breast tumors as either benign or malignant based on clinical features extracted from digitized biopsy data. By utilizing logistic regression, the project aims to automate the diagnostic process, thereby improving the speed, accuracy, and consistency of breast cancer detection. The goal is to reduce reliance on time-consuming and subjective manual diagnosis, provide a scalable solution that can assist healthcare professionals, especially in resource-limited settings, and ultimately support early detection efforts that can significantly improve patient outcomes. This project focuses on creating a practical, transparent, and efficient diagnostic tool that can serve as a foundation for further advancements in medical decision support systems.

1.3.1 Proposed System:

The proposed system is designed to automate the classification of breast tumours as benign or malignant using a supervised machine learning approach. It leverages the logistic regression algorithm, a widely accepted and interpretable method for binary classification tasks.

The system workflow begins with data acquisition, where the Wisconsin Breast Cancer Dataset is loaded directly from the scikit-learn library. Data preprocessing is performed to ensure the dataset is ready for modelling. This includes verifying the absence of missing values, standardizing feature scales using normalization techniques like Standard Scaler, and splitting the dataset into training and testing subsets. Data visualization techniques, using libraries such as Matplotlib and Seaborn, are employed to explore feature distributions and identify important patterns in the dataset.

After preprocessing, a logistic regression model is trained on the training data. Logistic regression is chosen for its simplicity, computational efficiency, and transparency in showing how each feature influences the prediction. Once trained, the model is evaluated based on accuracy and confusion matrix metrics, providing a clear measure of its performance in distinguishing between benign and malignant cases.

Python is the primary programming language used, supported by libraries such as scikit-learn, pandas, NumPy, Matplotlib, and Seaborn.

The environment chosen for development and execution is Google Colab, providing cloud-based access and ease of collaboration.

Overall, the proposed system provides a reliable, efficient, and interpretable tool for early breast cancer diagnosis. While it focuses on structured data and excludes the use of raw Medical imaging, it lays a strong foundation for future enhancements.

1.3.2 Advantages of the proposed system:

The proposed system offers several key advantages that make it an effective tool for breast cancer diagnosis. One of the major strengths is its automation of the classification process, which significantly reduces the time and manual effort needed for diagnosis. By providing quick and accurate results, the system supports healthcare professionals in making faster decisions, which is critical in the early detection and treatment of breast cancer. This can ultimately lead to improved patient survival rates and more efficient healthcare delivery.

Another important advantage is the system's interpretability. Since the model uses logistic regression, a simple and transparent algorithm, medical professionals can easily understand how the model reaches its predictions. This interpretability builds trust in the model's decisions, making it more acceptable for integration into real-world clinical workflows, where transparency and accountability are vital.

The system is also highly efficient and lightweight. It does not require complex hardware or large computational resources, making it accessible even in resource-constrained environments such as rural clinics or smaller healthcare facilities. This ensures that the benefits of machine learning can be extended beyond large hospitals to areas where medical expertise may be limited.

Additionally, the scalability and flexibility of the system allow for future expansion. While it currently focuses on logistic regression for binary classification, the system's architecture is designed to accommodate more advanced algorithms, larger datasets, or integration with other diagnostic tools if needed. This makes it a strong foundational project that can evolve with technological advancements.

Finally, the proposed system promotes better resource management by reducing the dependency on expert diagnosticians for routine evaluations. By assisting in preliminary diagnosis, it frees up specialist time for more complex cases, ultimately optimizing the overall efficiency of healthcare services.

1.4 Research

Numerous studies have explored the use of machine learning models for early detection of diseases such as breast cancer, given the critical importance of timely and accurate diagnosis. Research shows that algorithms like Decision Trees, Support Vector Machines (SVMs), Random Forests, K-Nearest Neighbours (KNN), and Neural Networks have been successfully applied to breast cancer classification tasks. These models have demonstrated the ability to learn patterns from complex clinical datasets and assist in improving diagnostic outcomes.

The Wisconsin Breast Cancer Dataset has emerged as one of the most widely used datasets for research in breast cancer classification. Several studies have utilized this dataset to benchmark the performance of different machine learning algorithms. For example, SVMs with non-linear kernels have been shown to achieve high classification accuracy by capturing complex patterns within the dataset. Similarly, Random Forests have been noted for their robustness against overfitting and their ability to handle noisy data effectively, making them strong candidates for medical data classification.

Despite the success of these complex models, simpler algorithms like Logistic Regression continue to play a crucial role in healthcare applications due to their interpretability and computational efficiency. Research indicates that when the dataset is linearly separable and features are properly pre-processed, logistic regression can perform competitively with more complex models. decision-making where understanding the rationale behind a diagnosis is critical.

Furthermore, studies have emphasized the importance of data preprocessing steps such as feature scaling, handling missing values, and proper dataset splitting to enhance model performance. Visualization tools like Matplotlib and Seaborn have also been employed in previous research to better understand feature distributions and relationships, aiding in feature selection and model evaluation.

Overall, the existing body of research supports the choice of logistic regression for this project, especially in scenarios where explainability, speed, and accessibility are prioritized over marginal gains in predictive accuracy. This project builds upon these findings a logistic regression-based breast cancer classification system, focusing on simplicity, reliability, and practical usability in real-world healthcare settings.

SYSTEM REQUIREMENTS

2.SYSTEM REQUIREMENTS

2.1 HARDWARE REQUIREMENTS

- **Processor** - Pentium–III
- **Speed** - 2.4GHz
- **RAM** - 512 MB (min)
- **Hard Disk** - 20 GB (min)
- **Keyboard** - Standard Keyboard
- **Monitor** - 15 VGA Color(min)

2.2 SOFTWARE REQUIREMENTS

Functional requirements for a secure cloud storage service are straightforward:

- The service should be able to store the user's data.
- The data should be accessible through any device connected to the Internet.
- The service should be capable of synchronizing the user's data between multiple devices (notebooks, smart phones, etc.);
- The service should preserve all historical changes(versioning);
- Data should be shareable with other users.
- The service should support SSO
- The service should be interoperable with other cloud storage services, enabling data migration from one CSP to another.
- Operating System: Windows
- Coding Language: Python 3.7
- Script
- Database

SYSTEM ANALYSIS

3. TECHNOLOGIES LEARNT

3.1 PYTHON

What is Python: -

Below are some facts about Python. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc. The biggest strength of Python is the huge collection of standard library which can be used for the following:

Machine Learning

GUI Applications (like Kivy, Tkinter, PyQt etc.)

Web frameworks like Django (used by YouTube, Instagram, Dropbox)

Image processing (like OpenCV, Pillow)

Web scraping (like Scrapy, BeautifulSoup, Selenium)

Test frameworks

Multimedia

Advantages of Python :-

Let's see how Python dominates over other languages.

- **Extensive Libraries**

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

- **Extensible**

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

- **Embeddable**

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

- **Improved Productivity**

The language's and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

Since Python forms the basis simplicity of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

- **Simple and Easy**

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

- **Readable**

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. These further aids the readability of the code.

- **Object-Oriented**

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

- **Free and Open-Source**

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform.

But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system- dependent features.

- **Interpreted**

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Advantages of Python Over Other Languages;-

- **Less Coding**

Almost all of the tasks done in Python require less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

- **Affordable**

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

- **Python is for Everyone**

Python code can run seamlessly across any platform — whether it's Linux, Mac, or Windows. Traditionally, programmers had to learn different languages for different tasks, but Python has broken that barrier. With Python, you can professionally build web applications, perform data analysis, dive into machine learning, automate repetitive tasks, scrape data from the web, and even develop games and powerful visualizations. Its extensive libraries and supportive community make it an all-rounder programming language, accessible to both beginners and professionals. Whether you're an aspiring developer, a data scientist, a researcher, or simply a curious learner, Python opens the door to endless possibilities, empowering innovation across every industry.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

- **Speed Limitations**

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

- **Weak in Mobile Computing and Browsers**

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

- **Design Restrictions**

As you know, Python is dynamically typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

- **Underdeveloped Database Access Layers**

Compared to more widely used technologies like JDBC (Java Data Base Connectivity) and ODBC (Open Data Base Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

- **Simple**

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python :-

What do the alphabet and the programming language Python have in common?

Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So, I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

3.2 MACHINE LEARNING

What is Machine Learning: -

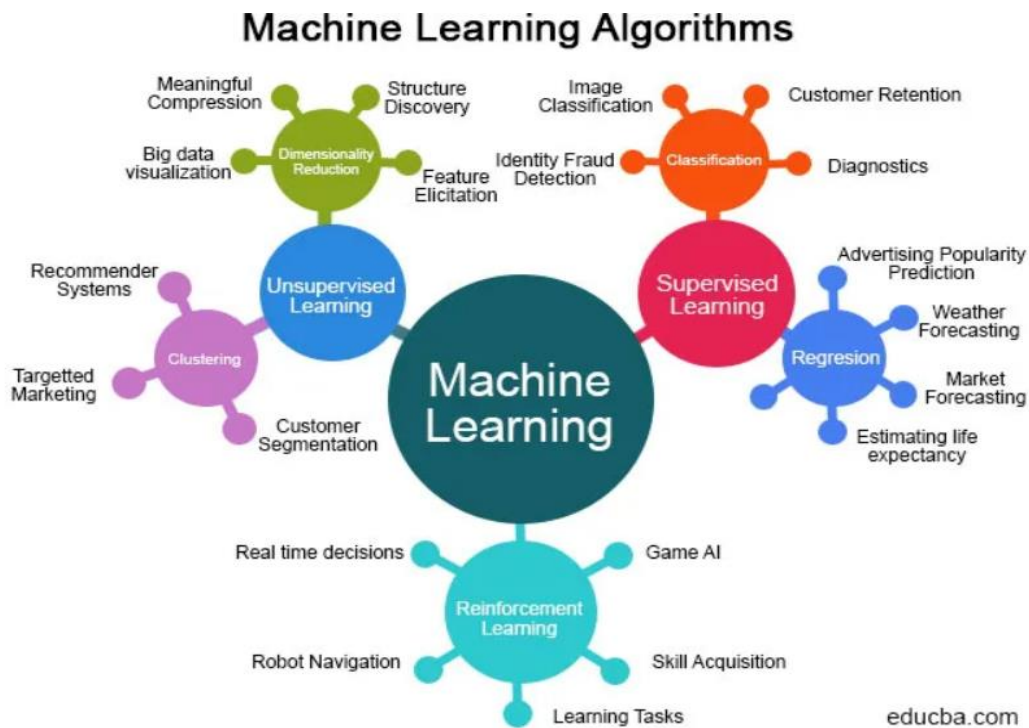
Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data.

Fundamentally, machine learning involves building mathematical models to help us understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be

"learning" from the data.. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. Will leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :-

- At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.
- Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.
- Unsupervised learning involves modeling the features of a dataset without reference to any label and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.



Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machine Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind this is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

- **Quality of data** – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to problems related to data preprocessing and feature extraction.
- **Time-Consuming task** – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.
- **Lack of specialist persons** – As ML technology is still in its infancy stage, availability of expert resources is a tough job.
- **No clear objective for formulating business problems** – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.
- **Issue of overfitting & underfitting** – If the model is overfitting or underfitting, it cannot be represented well for the problem.
- **Curse of dimensionality** – Another challenge ML model faces is too many features of data points. This can be a real hindrance.
- **Difficulty in deployment** – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications for Machines Learning: -

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many complex real-world problems which cannot be solved with traditional approaches. Following are some real-world applications of ML –

- **Healthcare**

Machine learning is transforming healthcare by enabling early disease detection, personalized treatment plans, and predictive diagnostics. For example, algorithms can analyse medical images to detect cancer at early stages, often with accuracy comparable to experienced radiologists.

- **Finance**

In finance, machine learning is used for fraud detection, risk assessment, and algorithmic trading. Systems can quickly spot unusual transaction patterns that may indicate fraud, helping protect both consumers and institutions.

- **Retail**

Retailers use machine learning to personalize shopping experiences, optimize inventory, and predict customer behaviour. Recommendation systems, like those used by Amazon and Netflix, suggest products based on past preferences, boosting sales and customer satisfaction.

- **Transportation**

Self-driving cars are one of the most well-known machine learning applications in transportation. ML models help vehicles recognize traffic signs, detect pedestrians, and make real-time driving decisions, improving road safety and efficiency.

- **Agriculture**

Machine learning helps farmers optimize crop yields by predicting weather patterns, diagnosing plant diseases, and monitoring soil health. Smart farming technologies lead to more sustainable and efficient agricultural practices.

- **Education**

Machine learning personalizes learning by adapting educational content to each student's pace and style. Intelligent tutoring systems and automated grading tools also make learning more efficient and accessible.

- **Cybersecurity**

In cybersecurity, machine learning identifies threats like malware and phishing attacks by detecting unusual patterns in network traffic. It helps organizations respond to threats faster and more accurately.

- **Entertainment**

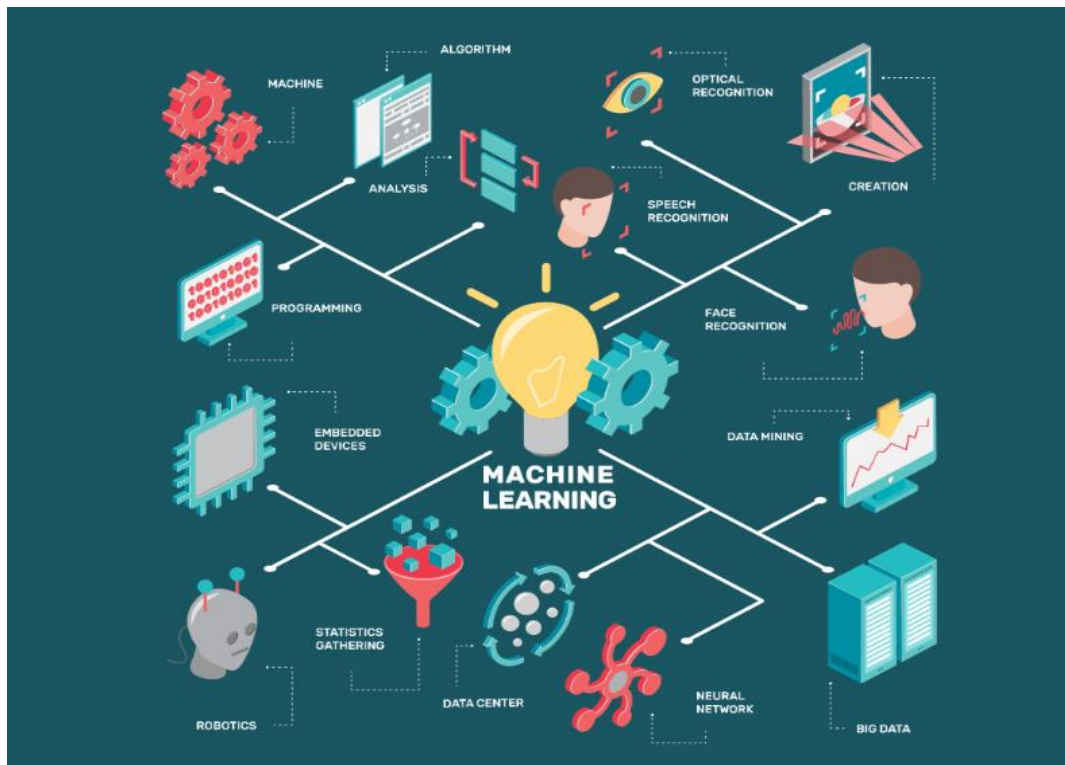
Streaming services like Spotify and YouTube use machine learning to recommend music and videos based on user preferences. It enhances user engagement by delivering highly relevant content.

- **Manufacturing**

In manufacturing, machine learning predicts equipment failures before they happen. Predictive maintenance reduces downtime, saves costs, and increases production efficiency.

- **Environmental Science**

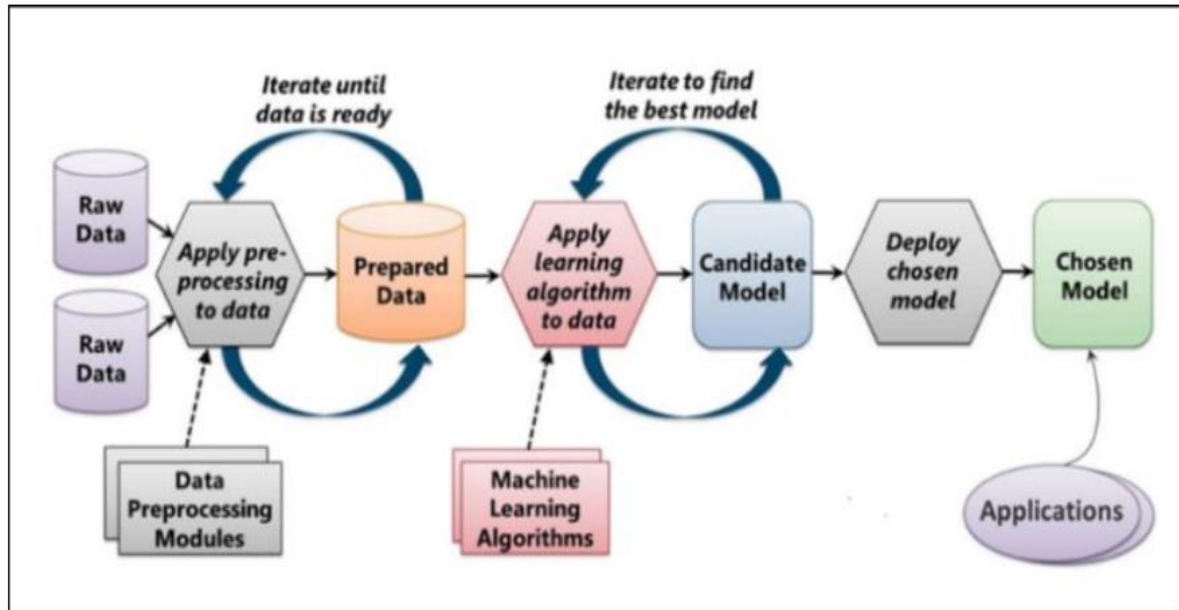
Machine learning aids in monitoring climate change, predicting natural disasters, and tracking wildlife. These models process vast amounts of environmental data to help.



How to Start Learning Machine Learning?

Arthur Samuel coined the term “Machine Learning” in 1959 and defined it as a “Field of study that gives computers the capability to learn without being explicitly programmed”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of \$146,085 per year.



How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

- **Learn Linear Algebra and Multivariate Calculus**

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on math as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

- **Learn Statistics**

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So, it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

- **Learn Python**

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So, if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as Fork Python available Free on GeeksforGeeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.

- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).
- **Supervised Learning**-Type of ML where the model is trained on labelled data (both inputs and outputs are known).
- **Unsupervised Learning**-Type of ML where the model tries to learn patterns from unlabelled data (no outputs provided)
- **Overfitting**-When a model learns the training data too well, including noise and outliers, and performs poorly on new, unseen data.
- **Underfitting**-When a model is too simple and cannot capture the underlying patterns in the data, resulting in poor performance on both training and testing data.
- **Loss Function**-A function that measures how far the model's predictions are from the actual outcomes. The model aims to minimize this loss.

Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabeled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabeled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So, the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

- **Easily identifies trends and patterns**

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

- **No human intervention needed (automation)**

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus software's; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

- **Continuous Improvement**

As ML algorithms gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

- **Handling multi-dimensional and multi-variety data**

Machine Learning algorithms are good at handling data that are multi-dimensional and multi- variety, and they can do this in dynamic or uncertain environments.

- **Wide Applications**

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning: -

- **Data Acquisition**

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

- **Time and Resources**

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

- **Interpretation of Results**

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

Python Development Steps: -

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included exception handling, functions, and the core data types of lists, dict, str and others. It was also object oriented and had a module system.

Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehension, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one-- obvious way to do it." Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted .
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "/" can be used to have the "old" behavior.
- Text Vs. Data Instead of Unicode Vs. 8-bit

Purpose: -

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. Python features a dynamic type of system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP. Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking. Its selection is attributed to several key factors, including its simplicity and readability, which facilitate efficient development. Furthermore, Python boasts an extensive collection of libraries that provide pre-built tools and functions, streamlining various tasks within the project.

3.3 Modules used in Project: -

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

- Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.
- It is the fundamental package for scientific computing with Python. It contains various features including these important ones:
 - A powerful N-dimensional array object
 - Sophisticated (broadcasting) functions
 - Tools for integrating C/C++ and Fortran code
 - Useful linear algebra, Fourier transform, and random number capabilities
 - Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It made very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, et

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality

figures in a variety of hardcopy formats and interactive environments across platforms.

Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type of system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to

another area where Python excels.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high- level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system is a Windows 64-bit operating system. So, the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet [here](#). The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GP0
Copied source tarball	Source release		68111671e5b2db4aef7b9ab01b0f9be	23017663	36
XZ compressed source tarball	Source release		d33e4a8e6097051c2eca45ee3604893	17331432	36
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583da9f1e442c8a8ce08e6	34898416	36
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217e45773b9e4e936b243f	28882845	36
Windows help file	Windows		863999573a2c98b2e38c8ade0b4f7ad2	8131761	36
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9809c3c7fd9ee3b8a8e82184a8f72ba2	7504391	36
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0a7f6d6d63043a83e563400	26882948	36
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28c31c908b8d73a8e651a3b831b4bd2	1362904	36
Windows x86 embeddable zip file	Windows		9fab38d188a41879fd8a4122574139d9	6741628	36
Windows x86 executable installer	Windows		33c802942a5448a3d846147e394789	25663848	36
Windows x86 web-based installer	Windows		1b670cf6d317d82c30883ea371d87c	1324608	36

- To download **Windows 32-bit python**, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download **Windows 64-bit python**, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

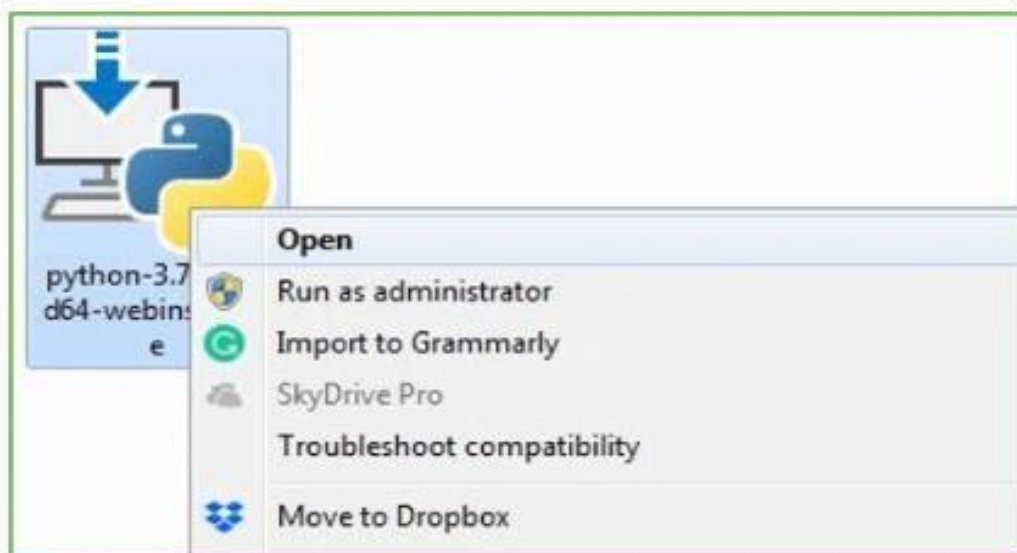
Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation



Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, make sure to add a tick on Add Python 3.7 to PATH.

Step 3: Click on Install NOW After the installation is successful. Click on Close.



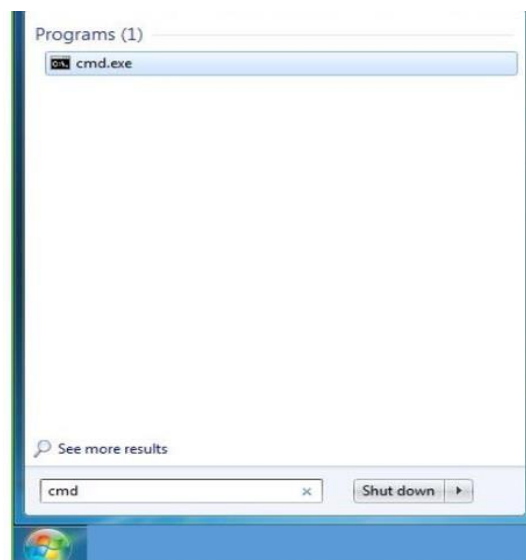
With these above three steps on Python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

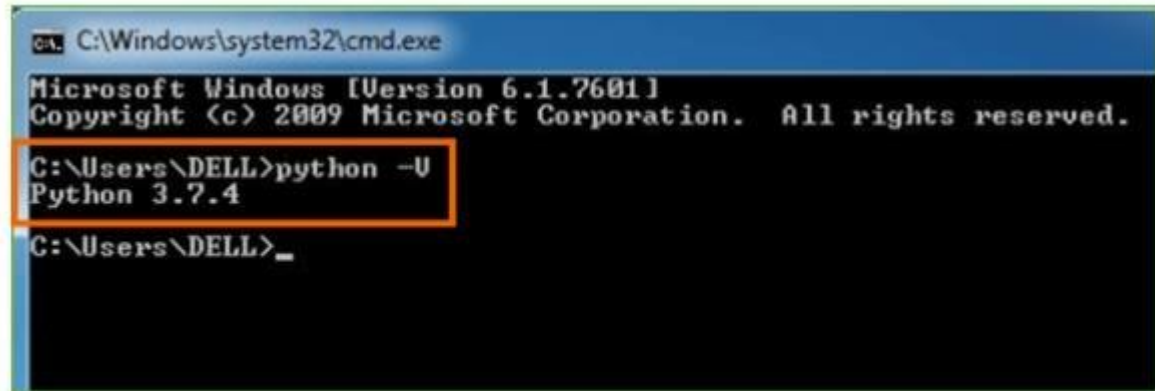
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type `python -V` and press Enter.



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\DELL>python -V
Python 3.7.4

C:\Users\DELL>_
```

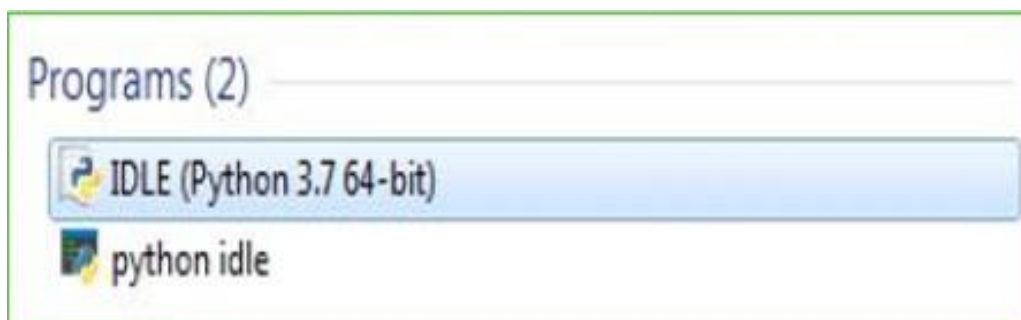
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

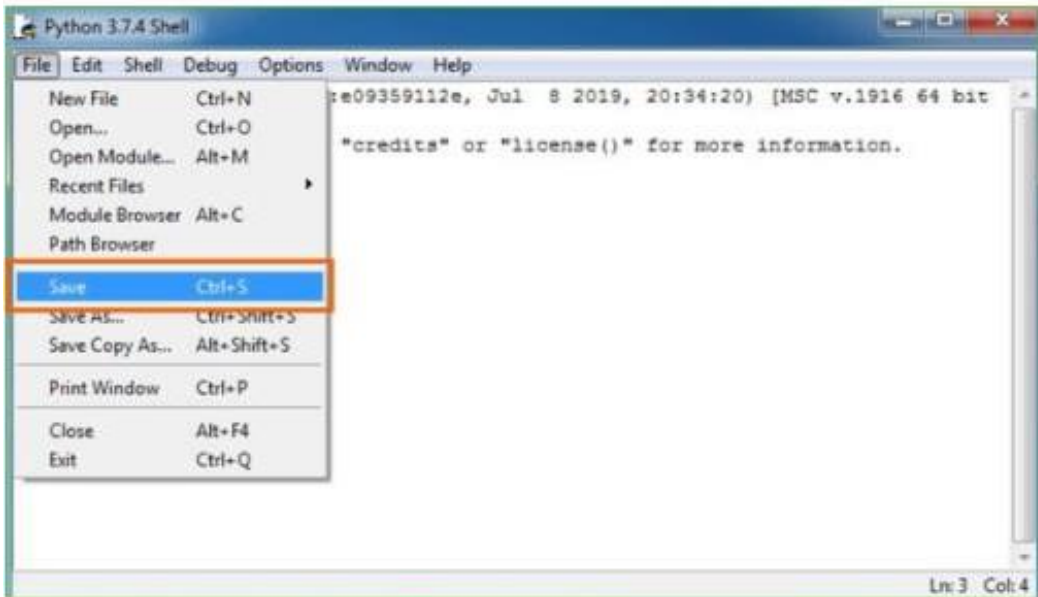
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”



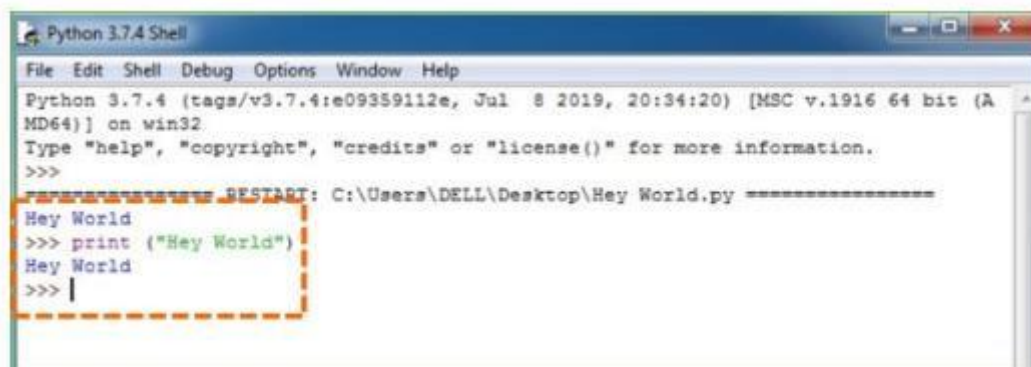
Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files Hey World.

Step 6: Now for e.g. enter print (“Hey World”) and Press Enter.



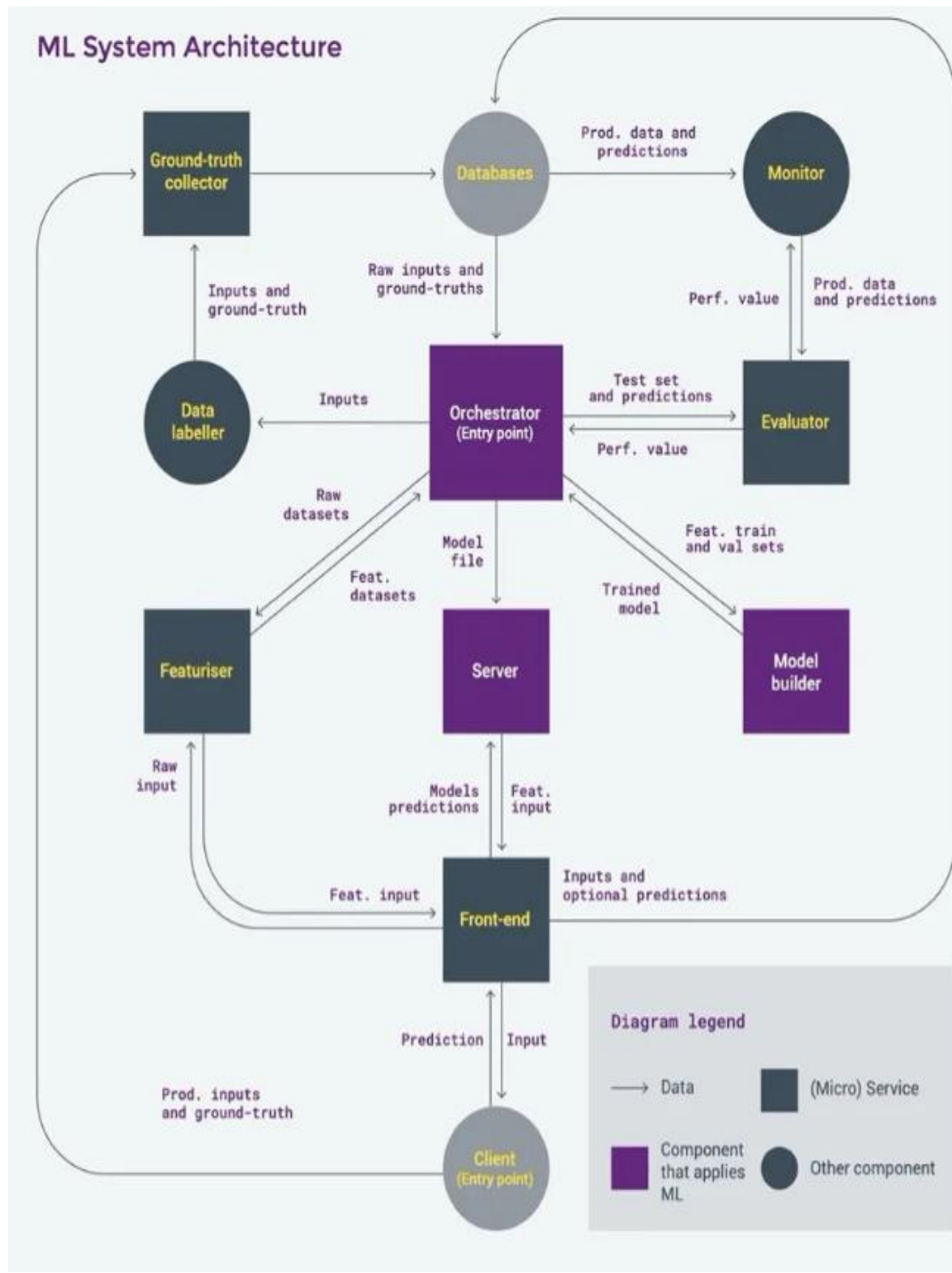
You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.

Note: Unlike Java, Python doesn't need semicolons at the end of the statements otherwise it won't work.

SYSTEM DESIGN

4.SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE



Module Description

In this project, feature selection plays a crucial role in reducing data dimensionality and enhancing the performance of the machine learning model. Two primary methods are employed for feature selection: filter methods and wrapper methods. Filter methods select features based on their statistical relationship with the output variable, independent of any machine learning algorithm. In contrast, wrapper methods evaluate different subsets of features by training a model and selecting the subset that provides the best performance.

Feature selection is an important part in machine learning to reduce data dimensionality and extensive research is carried out for a reliable feature selection method. The wrapper method finds a subset of features by measuring the usefulness of a subset of feature with the dependent variable. Hence filter methods are independent of any machine learning algorithm whereas in wrapper method the best feature subset used to train the model. The classifier considers the subset of feature with which the classification algorithm performs the best. To find the subset, the evaluator uses different search techniques like depth first search, random search, breadth first search or hybrid search. The filter method uses an attribute evaluator along with a ranker to rank all the features in the dataset. Here one feature is omitted at a time that has lower ranks and then sees the predictive accuracy of the classification algorithm. Weights or rank put by the ranker algorithms are different than those by the classification algorithm. The wrapper method is useful for machine learning tests whereas filter method is suitable for data mining test because data mining has thousands of millions of features.

The project focuses on breast cancer classification by using feature selection techniques such as filter methods and wrapper methods to improve model accuracy and reduce data complexity. After selecting relevant features, the data is preprocessed by cleaning, normalizing, and visualizing it using libraries like Matplotlib and Seaborn. A Logistic Regression model is trained on the processed data due to its simplicity and interpretability for binary classification. The model is evaluated using metrics such as accuracy, precision, recall, and F1-score. Finally, a predictive system is built to classify new patient data as benign or malignant. The project is developed using Python 3.7 in the Google Colab environment with libraries like scikit-learn, pandas, and NumPy, ensuring efficiency, transparency, and scalability.

Algorithms used in this project: -

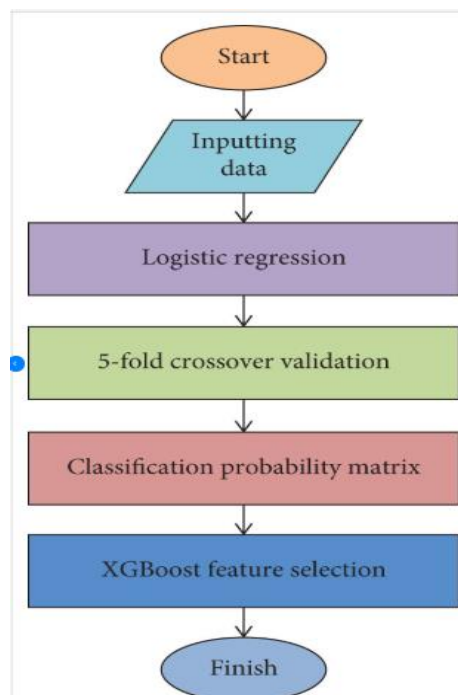
The main machine learning algorithm used in this project is Logistic Regression. Logistic Regression is a supervised learning algorithm commonly used for binary classification problems — in this case, classifying breast tumours as either benign or malignant. It works by modelling the probability that a given input belongs to a particular class based on a set of input features. Logistic regression is preferred here due to its simplicity, efficiency, and interpretability, making it ideal for healthcare-related decision support systems.

Additionally, for feature selection, two techniques are used:

- **Filter Methods:** These select features based on statistical tests that measure the relevance of each feature independently of any machine learning model.
- **Wrapper Methods:** These involve training and evaluating models on different subsets of features to select the best-performing subset.

Both methods help in reducing the number of input variables, improving model performance, and avoiding overfitting.

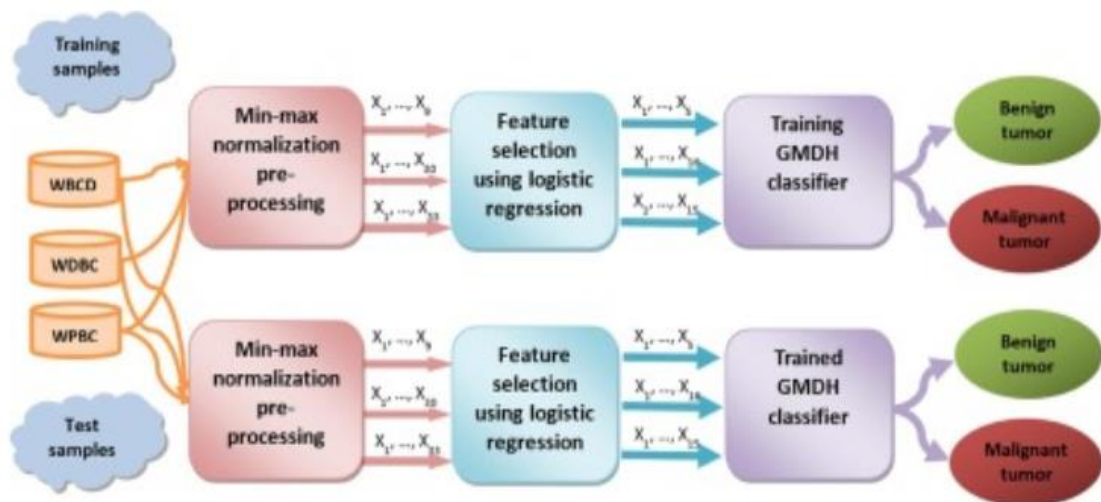
Supporting this process, data preprocessing techniques like normalization (using StandardScaler) and data splitting (using `train_test_split`) are applied, and TF-IDF Vectorization is used when dealing with text data, although primarily structured numerical data is handled in this project.



4.2 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

During the training phase of the project, the cleaned and pre-processed breast cancer dataset is divided into training and testing samples using the `train_test_split` function from `scikit-learn`. Typically, 70–80% of the data is allocated for training, while the rest is reserved for testing. The training samples are used to fit the Logistic Regression model, allowing it to learn the relationship between the input clinical features. The model adjusts its internal parameters (coefficients) during training to minimize the classification error.



In this breast cancer classification project, feature selection with logistic regression focuses on identifying the most important clinical features that strongly influence whether a tumor is malignant. Features such as mean radius, mean texture, mean perimeter, and mean smoothness often show strong correlations with malignancy. By selecting only the most relevant features, the logistic regression model can more accurately learn the patterns that distinguish malignant tumors from benign ones. During training, the model assigns higher weights (coefficients) to features that are more strongly associated with malignant tumors. As a result, when new patient data is input, the logistic regression model uses these key features to calculate the probability of malignancy, providing a reliable and interpretable output that classifies tumors as malignant or benign.

4.3 PRELIMINARY INVESTIGATION

The preliminary investigation for this project began by identifying the urgent need for a reliable, automated system to classify breast tumors as benign or malignant. Manual diagnostic methods are time-consuming, prone to human error, and heavily dependent on expert availability, particularly in resource-limited settings. Recognizing these challenges, the project aimed to develop a machine learning solution using logistic regression that could assist healthcare professionals in making faster and more accurate diagnoses. A feasibility study was conducted to evaluate the operational, economic, and technical viability of the system. It was found that the project could be implemented with minimal computational resources, using existing libraries and datasets like the Wisconsin Breast Cancer Dataset. With Python and scikit-learn as the primary development tools, and Google Colab as the environment, the project was deemed both cost-effective and technically feasible. Thus, the project was approved for further development, ensuring it could contribute meaningfully to early breast cancer detection and improved patient care.

REQUEST CLARIFICATION

The project titled "Breast Cancer Classification Using Machine Learning" focuses on building a machine learning model to classify breast tumours as either benign (non-cancerous) or malignant (cancerous). The system is designed to assist healthcare professionals by automating part of the diagnostic process, which traditionally is manual, time-consuming, and sometimes prone to errors. For this purpose, the project uses the Wisconsin Breast Cancer Dataset, which contains clinical features extracted from fine needle aspirate (FNA) images of breast masses. These features help the model learn patterns that differentiate between benign and malignant tumours.

The primary technology stack for the project includes Python 3.7 as the programming language, with Google Colab as the development environment for easy collaboration and cloud-based computing. Several libraries were employed: NumPy for numerical computations, Pandas for data manipulation, Matplotlib and Seaborn for data visualization, and scikit-learn for model building and evaluation. The project uses Logistic Regression, a simple but highly effective algorithm for binary classification tasks. Logistic Regression was chosen because of its interpretability, speed, and proven reliability in healthcare applications where explaining the results to medical professionals is important.

One of the major advantages of this system is its speed and simplicity, making it suitable for deployment even in resource-limited environments. Its interpretability is a strong point, helping doctors and healthcare workers trust and understand the model's predictions. It is also lightweight and scalable, allowing it to be easily expanded with more complex models in the future if needed. Furthermore, it reduces the manual workload of medical experts, enabling quicker diagnosis and improved patient outcomes.

FEASIBILITY ANALYSIS

An important outcome of preliminary investigation is the determination that the system request is feasible. This is possible only if it is feasible within limited resources and time. The different feasibilities that have to be analyzed are

- Operational Feasibility
- Economic Feasibility
- Technical Feasibility
- **Operational Feasibility**

Operational Feasibility deals with the study of prospects of the system to be developed. This system operationally eliminates all the tensions of the admin and helps him in effectively tracking the project progress. This kind of automation will surely reduce the time and energy which previously consumed in manual work. Based on the study, the system has proved to be operationally feasible.

- **Economic Feasibility**

Economic Feasibility or Cost-benefit is an assessment of the economic justification for a computer-based project. As hardware was installed from the beginning & for lots of purposes thus the cost on project of hardware is low.

The Virtual Private Network is to be developed using the existing resources of the organization. So, the project is economically feasible.

- **Technical Feasibility:**

According to Roger S. Pressman, Technical Feasibility is the assessment of the technical resources of the organization. The organization needs IBM compatible

machines with a graphical web browser connected to the Internet and Intranet. The system is developed for platform independent environment. Java Server Pages, JavaScript, HTML, SQL server and WebLogic Server are used to develop the system. Technical feasibility has been carried out. The system is technically feasible for development and can be developed with the existing facility.

REQUEST APPROVAL

Not all request projects are desirable or feasible. Some organizations receive so many project requests from client users that only few of them are pursued. However, those projects that are both feasible and desirable should be put onto schedule. After a project request is approved, its cost, priority, completion time and personnel requirement are estimated and used to determine where to add it to any project list. Truly speaking, with the approval of those above factors, development works can be launched.

SOURCE CODE

5.SOURCE CODE

Step1:- Importing the Dependencies

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

import numpy as np

- Imports the NumPy library, which is used for numerical computations.
- Provides support for arrays, mathematical functions, and linear algebra operations.
- For handling numerical operations on feature values (e.g., converting data into arrays for model training).

import pandas as pd

- Imports the Pandas library for data manipulation and analysis.
- To load, clean, and manipulate the breast cancer dataset in table format using DataFrames.
- Helps create and manipulate **DataFrames** to organize cancer data in tabular format.

import matplotlib.pyplot as plt

- Imports Matplotlib, a 2D plotting library.
- To visualize data distributions, correlation plots, or model performance.
- Tumor feature distributions, and model accuracy trends (like confusion matrix, ROC curve).

import seaborn as sns

- Imports Seaborn, a data visualization library built on top of Matplotlib.
- Helps create more attractive and informative statistical graphics, such as heatmaps or distribution plots for exploratory data analysis (EDA).
- Used to plot heatmaps and correlation matrices for features (e.g., how radius, texture, etc., relate to malignancy).

import sklearn.datasets

- Used to load the breast cancer dataset from sklearn.datasets.

- Contains built-in datasets like the breast cancer, iris, digits datasets, etc., useful for experimentation and learning.

from sklearn.model_selection import train_test_split

- Imports a utility function from scikit-learn to split datasets.
- Used to divide the dataset into training and testing sets. For example, 80% of the data can be used to train the model and 20% to evaluate it.

from sklearn.linear_model import LogisticRegression

- Imports the Logistic Regression algorithm.
- Used to create and train a logistic regression model for classification tasks.
- To apply Logistic Regression, a supervised classification algorithm, to predict whether a tumor is malignant or benign.

from sklearn.metrics import accuracy_score

- Imports various evaluation metrics to assess model performance.
- Used to measure the accuracy of the model's predictions.
- **accuracy_score**: Measures how many predictions were correct out of total predictions. Used to evaluate how well the model performed by calculating the proportion of correct predictions.

Step2:- Data Collection & Pre-Processing

```
# loading the data from sklearn
breast_cancer_dataset = sklearn.datasets.load_breast_cancer()

print(breast_cancer_dataset)
```

- To load the Breast Cancer Wisconsin dataset from the scikit-learn library into your Python program. This dataset is commonly used for training and testing machine learning models that perform binary classification (to predict whether a tumor is benign or malignant).
- This line is to provide an easy way to access a ready-to-use dataset without the need to manually download or clean data. After running this line, you get a structured object that contains:

data: the input features (e.g., size, shape of the tumor),

target: the output labels (0 for malignant, 1 for benign),

feature_names: the names of the clinical features,

DESCR: a detailed description of the dataset.

```
# loading the data to a data frame
data_frame = pd.DataFrame(breast_cancer_dataset.data, columns = breast_cancer_dataset.feature_names)
```

```
# print the first 5 rows of the dataframe
data_frame.head()
```

- Is to convert the raw data from the breast cancer dataset into a structured Pandas DataFrame. A DataFrame is like a table (similar to an Excel sheet) where data is organized in rows and columns, making it much easier to view, explore, and manipulate.
- Create a DataFrame where each row represents a sample (a patient's tumor),
- Each column represents a clinical feature (like mean radius, texture, area, smoothness, etc.),
- Allow you to easily perform data analysis, visualization, and preprocessing using Pandas functions like `head()`, `.describe()`, `.info()`, and many others.

```
# adding the 'target' column to the data frame
data_frame['label'] = breast_cancer_dataset.target
```

```
# print last 5 rows of the dataframe
data_frame.tail()
```

- Is to add a new column called 'label' to the existing DataFrame. This new column stores the target values from the breast cancer dataset, where each value indicates whether the tumor is benign (1) or malignant (0).
- Combine the input features (like mean radius, texture, perimeter, etc.) with their corresponding output label (benign or malignant) in the same table.
- Make it easier to work with the full dataset because now both the features and the label are together.
- Prepare the data conveniently for training, testing, visualization, or any kind of analysis.

```
# number of rows and columns in the dataset
data_frame.shape
```

- Is to check the size of the dataset — that is, how many rows and how many columns are in the DataFrame
- Quickly find out the number of samples (rows) and number of features + label (columns) in your dataset.
- Understand the dimension of your data before applying any machine learning techniques.

Output:-

(569, 31)

569 = number of rows → meaning there are 569 samples (each row is a patient/data point)

31 = number of columns → meaning there are 31 features (attributes) for each sample.

In the context of the Wisconsin Breast Cancer Dataset, the 31 columns typically include:

30 features (like mean radius, mean texture, mean smoothness, etc.).

1 label column (the diagnosis, e.g., malignant or benign).

```
# getting some information about the data
data_frame.info()
```

- Is to display important information about the DataFrame, such as the column names, data types, non-null values, and memory usage.
- Quickly check if there are any missing (null) values in the dataset.
- See the data type of each column (e.g., float64 for numbers, int64 for labels).
- Understand the structure of the dataset before proceeding to data cleaning, preprocessing, or modelling.
- Identify if any data needs conversion (for example, changing object types to numeric if needed).

Output:-

The output shown is from the `data_frame.info()` function in pandas, which provides a concise summary of the DataFrame. It reveals that the DataFrame belongs to the `pandas.core.frame.DataFrame` class and contains a total of 569 entries, with row indices ranging from 0 to 568. This means there are 569 rows in the dataset. There are 31 columns in total, each representing a different feature or attribute of the data.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                           569 non-null    float64
1   mean texture                           569 non-null    float64
2   mean perimeter                         569 non-null    float64
3   mean area                             569 non-null    float64
4   mean smoothness                        569 non-null    float64
5   mean compactness                       569 non-null    float64
6   mean concavity                         569 non-null    float64
7   mean concave points                   569 non-null    float64
8   mean symmetry                          569 non-null    float64
9   mean fractal dimension                 569 non-null    float64
10  radius error                           569 non-null    float64
11  texture error                           569 non-null    float64
12  perimeter error                        569 non-null    float64
13  area error                             569 non-null    float64
14  smoothness error                       569 non-null    float64
15  compactness error                      569 non-null    float64
16  concavity error                        569 non-null    float64
17  concave points error                   569 non-null    float64
18  symmetry error                         569 non-null    float64
19  fractal dimension error                 569 non-null    float64
20  worst radius                           569 non-null    float64
21  worst texture                           569 non-null    float64
22  worst perimeter                         569 non-null    float64
23  worst area                             569 non-null    float64
24  worst smoothness                       569 non-null    float64
25  worst compactness                      569 non-null    float64
26  worst concavity                        569 non-null    float64
27  worst concave points                   569 non-null    float64
28  worst symmetry                          569 non-null    float64
29  worst fractal dimension                 569 non-null    float64
30  label                                  569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB

```

```

# checking for missing values
data_frame.isnull().sum()

```

- Is to check for missing (null) values in each column of the Data Frame.
- Find out which columns have missing or empty values.
- Know how many missing values are present in each column.
- Decide if you need to handle missing data (e.g., by removing rows, filling missing values, or using imputation techniques).
- Ensure the dataset is clean and complete before training the machine learning model,

because missing data can cause errors or reduce model accuracy.

Output :-

.isnull() checks for missing values (i.e., NaN) in the dataset.

.sum() adds up the missing values column-wise.

Here, for every column (like mean radius, mean texture, mean perimeter, etc.), the value is 0.

Meaning: There are no missing values in any column of your dataset.

mean area	0
mean smoothness	0
mean compactness	0
mean concavity	0
mean concave points	0
mean symmetry	0
mean fractal dimension	0
radius error	0
texture error	0
perimeter error	0
area error	0
smoothness error	0
compactness error	0
concavity error	0
concave points error	0
symmetry error	0
fractal dimension error	0
worst radius	0
worst texture	0
worst perimeter	0
worst area	0
worst smoothness	0
worst compactness	0
worst concavity	0
worst concave points	0
worst symmetry	0
worst fractal dimension	0
label	0

dtype: int64

```
# statistical measures about the data
data_frame.describe()
```

- Quickly summarizes your dataset by calculating statistical measures for each column.
- Gives you a snapshot of the data distribution — super useful for exploratory data analysis (EDA).
- Gives quick summary statistics of a DataFrame.
- Works mainly on numeric columns by default.
- Shows: count, mean, std, min, 25%, 50%, 75%, max.
- Useful for exploring data fast

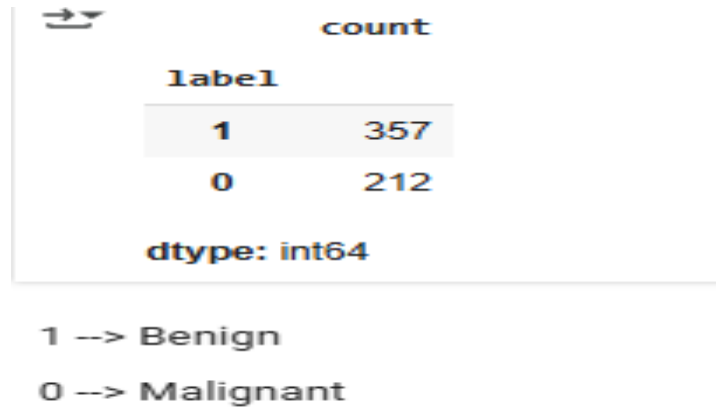
Output:-

The output of `data_frame.describe()` provides a statistical summary of the dataset. It shows key metrics like the count, mean, standard deviation (std), minimum (min), 25th percentile (25%), median (50%), 75th percentile (75%), and maximum (max) for each feature. From the output, we can see that all columns have 569 non-null entries, confirming no missing data. The mean, standard deviation, and range (min to max) values highlight that different features have different scales. This indicates that scaling or normalization might be necessary before applying machine learning models to ensure better performance.

```
# checking the distribution of Target Variable
data_frame['label'].value_counts()
```

- Counts how many times each unique value appears in the label column.
- Mainly used to check class distribution for the target variable (especially important in classification problems).
- Groups by unique values.
- Counts each group.
- Can normalize to get percentages (if `normalize=True`)

Output:-



The screenshot shows a Jupyter Notebook cell with a pivot table and a legend. The pivot table has 'label' as the index and 'count' as the column. The data is as follows:

label	count
1	357
0	212

Below the table, it says 'dtype: int64'. At the bottom, there is a legend: '1 -> Benign' and '0 -> Malignant'.

Label 1 (Benign): 357 samples

- Label 0 (Malignant): 212 samples
- Total samples: $357 + 212 = 569$
- Your dataset is imbalanced, with more benign cases (62.7%) than malignant cases (37.3%).
- Class imbalance can affect model performance, particularly if the model becomes biased toward predicting the majority class (benign).

```
data_frame.groupby('label').mean()
```

- Groups the data by the label column.
- Calculates the mean (average) of other numeric columns for each label group.
- Useful to compare feature averages across different classes (example: "Yes" vs "No").
- First group the DataFrame by the given column.
- Then apply `.mean()` to each group for all numeric columns.

Step3:- Separating the features and target

```
X = data_frame.drop(columns='label', axis=1)
Y = data_frame['label']
```

- **X** = input features (everything except 'label')
- **Y** = target variable ('label' column)
- **X-Features**: all columns except 'label'. (Input data for your model)

- Y-Target variable: only the 'label' column. (What you want to predict)

```
print(X)
```

Output:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.30010	0.14710	0.2419	
1	0.07864	0.08690	0.07017	0.1812	
2	0.15990	0.19740	0.12790	0.2069	
3	0.28390	0.24140	0.10520	0.2597	
4	0.13280	0.19800	0.10430	0.1809	
..	...	...	...	...	
564	0.11590	0.24390	0.13890	0.1726	
565	0.10340	0.14400	0.09791	0.1752	
566	0.10230	0.09251	0.05302	0.1590	
567	0.27700	0.35140	0.15200	0.2397	
568	0.04362	0.00000	0.00000	0.1587	

	mean fractal dimension	...	worst radius	worst texture	\
0	0.07871	...	25.380	17.33	
1	0.05667	...	24.990	23.41	
2	0.05999	...	23.570	25.53	
3	0.09744	...	14.910	26.50	
4	0.05883	...	22.540	16.67	
..	...	...	...	...	
564	0.05623	...	25.450	26.40	
565	0.05533	...	23.690	38.25	
566	0.05648	...	18.980	34.12	
567	0.07016	...	25.740	39.42	
568	0.05884	...	9.456	30.37	

	worst perimeter	worst area	worst smoothness	worst compactness	\
0	184.60	2019.0	0.16220	0.66560	
1	158.80	1956.0	0.12380	0.18660	
2	152.50	1709.0	0.14440	0.42450	
3	98.87	567.7	0.20980	0.86630	
4	152.20	1575.0	0.13740	0.20500	
..	...	...	...	...	
564	166.10	2027.0	0.14100	0.21130	
565	155.00	1731.0	0.11660	0.19220	
566	126.70	1124.0	0.11390	0.30940	
567	184.60	1821.0	0.16500	0.86810	
568	59.16	268.6	0.08996	0.06444	

	worst concavity	worst concave points	worst symmetry	\
0	0.7119	0.2654	0.4601	
1	0.2416	0.1860	0.2750	
2	0.4504	0.2430	0.3613	
3	0.6869	0.2575	0.6638	
4	0.4000	0.1625	0.2364	

```
print(Y)
```

Output:

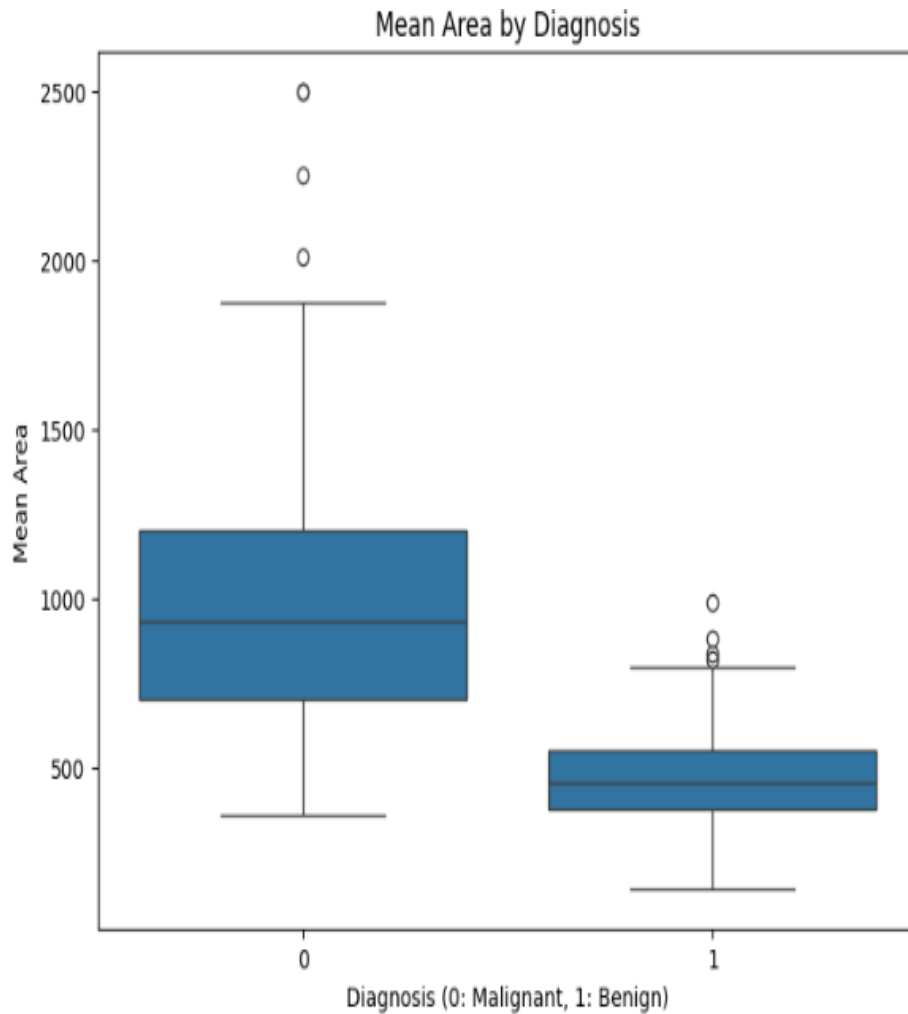
```
0      0
1      0
2      0
3      0
4      0
..
564    0
565    0
566    0
567    0
568    1
Name: label, Length: 569, dtype: int64
```

Step4:- Splitting the data into training data & Testing data

```
#Box plot for a feature grouped by target variable (e.g., 'mean area' by 'label')
plt.figure(figsize=(8, 6))
sns.boxplot(x='label', y='mean area', data=data_frame)
plt.title('Mean Area by Diagnosis')
plt.xlabel('Diagnosis (0: Malignant, 1: Benign)')
plt.ylabel('Mean Area')
plt.show()
```

- To visually compare the distribution of 'mean area' for each 'label' group.
- Helps you see differences in spread, median, outliers between classes like Malignant and Benign.
- Common in EDA (Exploratory Data Analysis) before model building.

Output:



```
#Correlation matrix heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(data_frame.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

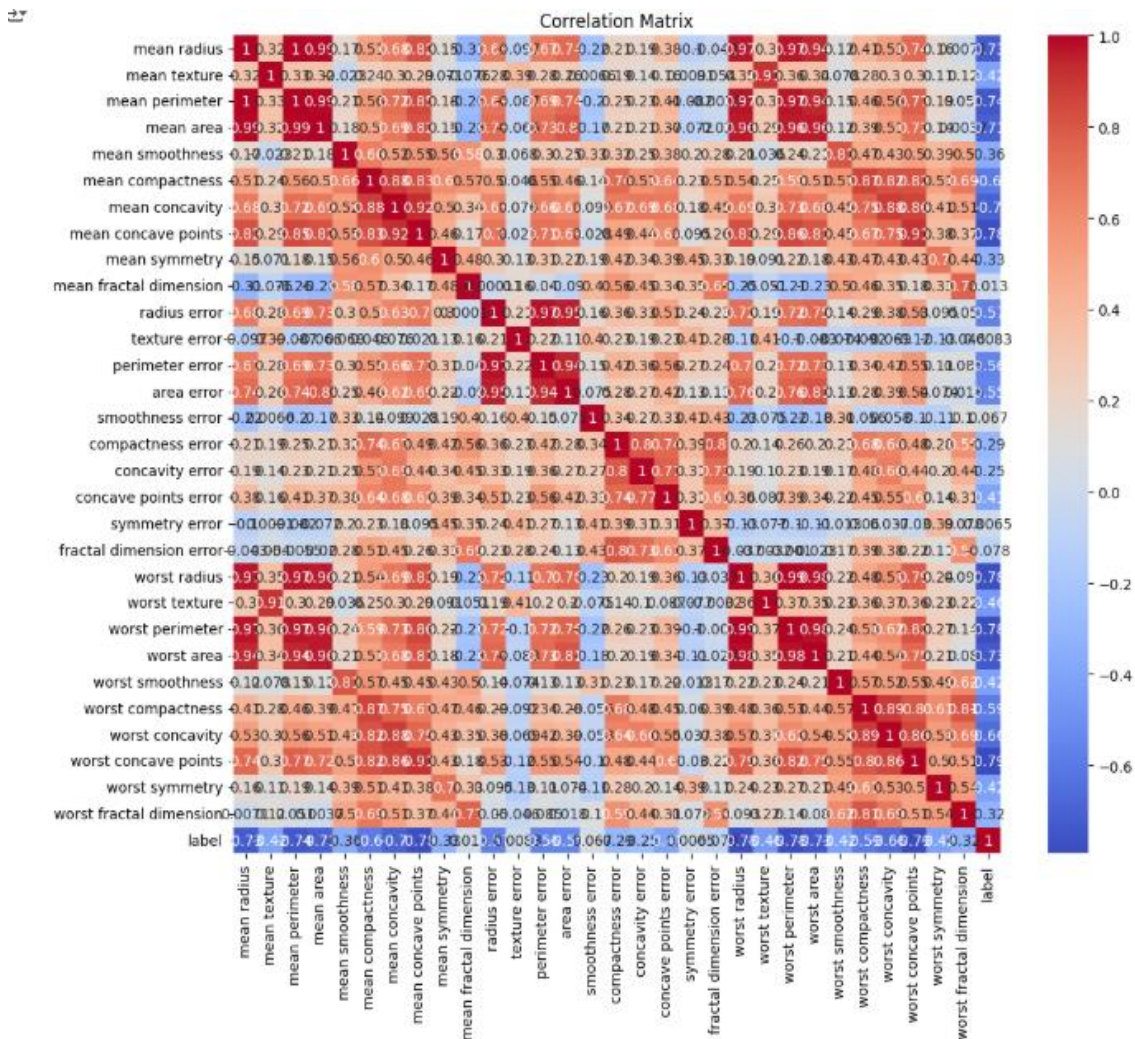
- Visualize correlations between all numeric features in the dataset.
- Shows how strongly features are related (positive or negative).
- Helps in feature selection, multicollinearity detection, and understanding data better.

1 → strong positive correlation (deep red)

-1 → strong negative correlation (deep blue)

0 → no correlation (neutral color)

Output:



The code generates a correlation matrix heatmap, which is a visual tool used to show how strongly different numerical features in a dataset are related to each other. It first sets the size of the heatmap to make it large enough for easy viewing. Then, using Seaborn's `heatmap()` function, it calculates the correlation between each pair of features in the dataset and displays them as a color-coded grid.

The `annot=True` argument ensures that the actual correlation values (ranging from -1 to 1) are shown within each cell, while the `cmap='coolwarm'` color map visually distinguishes positive correlations (shown in red) from negative ones (shown in blue). A title is added for clarity, and finally, the plot is displayed. This visualization helps identify which features are closely related and can guide decisions in feature selection or

dimensionality reduction for machine learning models.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

Output:

(569, 30) (455, 30) (114, 30)

- To split the data into training and testing parts.
- Train the model on training data, test on testing data.

Step5:- Model Training and Logistic Regression

```
model = LogisticRegression()
```

```
# training the Logistic Regression model using Training data
```

```
model.fit(X_train, Y_train)
```

- Creates an instance of the Logistic Regression model.
- No parameters passed here, so it uses default settings (like regularization type l2).
- **X_train:** Feature variables (like radius, texture, etc.).
- **Y_train:** Target labels (0 = benign, 1 = malignant).
- The model learns the relationship between features and the cancer type.

Output:

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
    LogisticRegression
```

```
LogisticRegression()
```

Step6:- Model Evaluation and Accuracy Score

```
# accuracy on training data
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(Y_train, X_train_prediction)
```

```
print('Accuracy on training data = ', training_data_accuracy)
```

```
Accuracy on training data = 0.9472527472527472
```

```
# accuracy on test data
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(Y_test, X_test_prediction)
```

```
print('Accuracy on test data = ', test_data_accuracy)
```

```
Accuracy on test data = 0.9298245614035088
```

Training Accuracy:

- Predicts outcomes for the training data (X_train) using the trained model.
- Compares these predictions (X_train_prediction) with the actual training labels (Y_train) using accuracy_score() to get training_data_accuracy.
- Prints the calculated training accuracy.

Test Accuracy:

- Predicts outcomes for the unseen test data (X_test) using the same model.
- Compares these predictions (X_test_prediction) with the actual test labels (Y_test) using accuracy_score() to get test_data_accuracy.
- **model:** The same trained machine learning model used for training data prediction.
- **.predict(X_test):** The predict() method is called with the feature matrix of the test data (X_test) as input.
- **X_test:** This variable holds the feature matrix of the test data. The test data is a separate dataset that was not used during the training of the model. It's used to evaluate the model's generalization ability on new, unseen data.
- **accuracy_score():** The same accuracy scoring function is used.
- **Y_test:** This variable holds the true labels or target values for the test data.
- **X_test_prediction:** This variable contains the model's predictions for the test data.
- **test_data_accuracy:** The calculated accuracy score for the test data is stored here.
- This score is a crucial indicator of how well the model is likely to perform on new,

real-world data.

- **'Accuracy on test data = '**: A string literal for clarity.
- **test_data_accuracy**: The accuracy score calculated for the test data will be printed.

Building a Predictive System

```
input_data = [(13.54,14.36,87.46,566.3,0.09779,0.08129,0.06664,0.04781,0.1885,0.05766,
               0.2699,0.7886,2.058,23.56,0.008462,0.0146,0.02387,0.01315,0.0198,0.0023,
               15.11,19.26,99.7,711.2,0.144,0.1773,0.239,0.1288,0.2977,0.07259)]

# change the input data to a numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the numpy array as we are predicting for one datapoint
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if (prediction[0] == 0):
    print('The Breast cancer is Malignant')
else:
    print('The Breast Cancer is Benign')
```

Output:-

```
[1]
The Breast Cancer is Benign
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739:
  warnings.warn(
```

The code converts this input into a NumPy array.

- The model (model) is then used to predict whether the tumor is malignant (cancerous) or benign (non-cancerous).
- Based on the model's prediction:
 - If the prediction is 0, it prints: **"The Breast cancer is Malignant"**.
 - Otherwise, it prints that it is **Benign**.

- This block of code interprets the numerical prediction and prints a human-readable diagnosis.
- **np.asarray():** This is a function from the NumPy library (np) that takes an input (in this case, the input_data tuple) and converts it into a NumPy array.
- **reshape(1, -1):** This is a method of the NumPy array.
- **1:** Specifies that the reshaped array should have one row.
- **-1:** Tells NumPy to automatically calculate the number of columns based on the original size of the array, ensuring all the data is included in the single row.
- **prediction [0]:** Accesses the first (and in this case, the only) element of the prediction array, which contains the predicted class label.
- **if (prediction [0] == 0)::** Checks if the predicted class label is 0. It's assumed that the model was trained such that one class (e.g., Malignant) is represented by 0.
- **print('The Breast cancer is Malignant'):** If the prediction is 0, this line prints that the model predicts the breast cancer is Malignant.
- **else::** If the condition in the if statement is false (meaning prediction[0] is not 0, and in a binary case, it's likely 1).
- **print('The Breast Cancer is Benign'):** If the prediction is not 0 (presumably 1), this line prints that the model predicts the breast cancer is Benign.

CONCLUSION

6.CONCLUSION

In conclusion, this project successfully developed a machine learning model, leveraging logistic regression, to accurately classify breast tumors as either benign or malignant based on clinical features. The model's performance demonstrates its potential as a valuable tool for assisting medical professionals in the crucial task of early breast cancer diagnosis. The high classification accuracy achieved, coupled with the inherent interpretability of logistic regression, positions this approach as particularly effective for initial screenings, enabling clinicians to understand the factors influencing the model's predictions. . This work not only provides an automated solution to streamline the diagnostic process, reducing time and effort, but also establishes a solid foundation for future enhancements. Further research can explore more complex models and integrate this technology into comprehensive decision-support systems, ultimately contributing to improved patient outcomes and advancements in breast cancer care.

Enhanced Diagnostic Capabilities

- The primary achievement of this project is the development of a machine learning model that can effectively classify breast tumours as either benign or malignant. The model provides a reliable and efficient tool that can assist medical professionals in making informed decisions, potentially leading to earlier detection and more timely interventions.

The Role of Logistic Regression

- The choice of logistic regression as the core algorithm is significant. While more complex machine learning models exist, logistic regression offers a balance of high accuracy, interpretability, and computational efficiency. In medical contexts, interpretability is crucial, as it allows healthcare providers to understand the reasoning behind the model's predictions, fostering trust and facilitating clinical adoption.

Automation and Efficiency

- The automation of breast cancer classification streamlines the diagnostic process, reducing the time and effort required for manual analysis. This is particularly important in resource-limited settings where access to specialists may be limited, and timely diagnosis can be challenging. The system's efficiency and scalability can contribute to improved healthcare delivery and patient outcomes.

FUTURE SCOPE

7.FUTURE SCOPE

Enhancing the Machine Learning Model:

Currently, logistic regression serves as a reliable baseline for the breast cancer classification task. However, future versions of the project could explore more advanced machine learning models such as Support Vector Machines (SVMs), Random Forests, and Neural Networks. These models often offer greater predictive accuracy and robustness, which could lead to improved clinical outcomes. Transitioning to more sophisticated algorithms would allow the system to capture complex patterns that logistic regression might miss.

Expanding and Improving the Dataset:

To further strengthen the model, expanding the dataset with a wider variety of samples is crucial. This includes gathering data from diverse populations and varying medical backgrounds. Additionally, employing feature engineering techniques—such as creating new informative features or enhancing existing ones—can significantly boost performance. Integrating other data modalities, like medical imaging or genetic information, would provide richer context and help the model make more accurate predictions.

Improving Model Explainability:

While logistic regression is valued for its interpretability, moving toward more complex models requires a continued focus on explainability. Techniques such as SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations) can help clarify model decisions, even when the algorithms are complex. Clear explanations foster trust among medical professionals, ensuring that AI-driven recommendations are understood and accepted.

Incorporating Unstructured Data:

Currently, the project focuses on structured data, such as tabular features. A promising avenue for growth is incorporating unstructured data types like medical images (e.g., mammograms or MRIs) and clinical notes. By combining structured and unstructured data, the model can provide a more comprehensive and accurate diagnosis, closely resembling the multi-faceted analysis that human doctors perform.

BIBLIOGRAPHY

8.BIBLIOGRAPHY

1. McHugh ML. The odds ratio: calculation, usage, and interpretation. *Biochem Med* 2009;19:120–6. <http://dx.doi.org/10.11613/BM.2009.011>.
2. Simundić AM. Bias in research. *Biochem Med* 2013;23:12 <http://dx.doi.org/10.11613/BM.2013.003>.
3. Mantel N, Haenszel W. Statistical aspects of the analysis of data from retrospective studies of disease. *J Natl Cancer Inst* 1959;22:719–48.
4. Bendel RB, Afifi AA. Comparison of Stopping Rules in Forward “Stepwise” Regression. *J Am Stat Assoc* 1977;72: 46–53.
5. Costanza MC, Afifi AA. Comparison of Stopping Rules in Forward Stepwise Discriminant Analysis. *J Am Stat Assoc* 1979;74:777–85. <http://dx.doi.org/10.1080/01621459.1979.10481030>.
6. Greenland S. Modeling and variable selection in epidemiologic analysis. *Am J Public Health* 1989;79:340–9. <http://dx.doi.org/10.2105/AJPH.79.3.340>.
7. Negrato CA, Cobas RA, Gomes MB. Temporal changes in the diagnosis of type 1 diabetes by diabetic ketoacidosis in Brazil: a nationwide survey. *Diabet Med* 2012;29:1142–7. <http://dx.doi.org/10.1111/j.1464-5491.2012.03590.x>.

BREAST CANCER
CLASSIFICATION USING
MACHINE LEARNING - Loukhya
Thota.docx

by A A

Submission date: 04-Jul-2025 02:19AM (UTC+0900)

Submission ID: 2701105132

File name: BREAST_CANCER_CLASSIFICATION_USING_MACHINE_LEARNING_- _Loukhya_Thota.docx (591.34K)

Word count: 3186

Character count: 20632

BREAST CANCER CLASSIFICATION USING MACHINE LEARNING

THOTA LOUKHYA

23MCA33, Student, MCA,

Department of Computer Science,

P.B. SIDDHARTHA COLLEGE OF ARTS AND SCIENCE

Vijayawada, A.P, India

loukhyathota123@gmail.com

ABSTRACT:

This project presents a machine learning-based solution for the classification of breast tumors as benign or malignant, using the Wisconsin Breast Cancer Dataset. Breast cancer is a major health concern and a leading cause of death among women globally, where early and accurate diagnosis significantly improves survival rates. Traditional diagnostic approaches are often time-consuming and prone to human error. To address this, the project employs a logistic regression model, chosen for its simplicity, interpretability, and effectiveness in binary classification.

The resulting model demonstrates high classification accuracy, making it a practical tool for assisting medical professionals in early diagnosis. Its light weight architecture and interpretability make it suitable for real-world clinical use, especially in resource-constrained environments. This project contributes toward enhancing medical diagnostic systems through accessible and reliable machine learning solutions.

KEYWORDS

Breast Cancer, Machine Learning, Logistic Regression, Classification, Binary Classification, Medical Diagnosis, Wisconsin Dataset, Python, Scikit-learn, Data Preprocessing, Feature Selection, Model Evaluation, Benign, Malignant, Predictive Modeling, Healthcare AI.

1. INTRODUCTION

Breast cancer is one of the most prevalent and life-threatening diseases affecting women worldwide. Early and accurate diagnosis plays a crucial role in improving survival rates and guiding timely treatment. However, traditional diagnostic methods—such as manual interpretation of biopsy results—are often time-consuming, subjective, and prone to human error, especially in regions lacking experienced medical professionals.

This project aims to develop a machine learning-based diagnostic system that can accurately classify breast tumors as either benign (non-cancerous) or malignant (cancerous). By leveraging clinical features extracted from digitized images of fine needle aspirate (FNA) of breast masses, the system uses a logistic regression algorithm to automate the classification process. Logistic

regression is chosen for its simplicity, computational efficiency, and interpretability making it well-suited for use in medical environments.

Manual diagnosis methods can be time-consuming and prone to error. This project aims to build a machine learning model that classifies tumors as benign or malignant using features derived from digitized images of a fine needle aspirate (FNA) of a breast mass. The model assists in reliable and efficient medical diagnosis.

Breast cancer continues to be one of the leading causes of death among women

globally. Early detection is vital for improving survival rates, yet traditional diagnostic methods such as manual analysis of biopsy results are often time-consuming, subjective, and heavily dependent on expert availability.

The motivation for this project stems from the need to develop a reliable, efficient, and interpretable diagnostic tool to assist medical practitioners in the early detection and classification of breast tumours. By leveraging machine learning specifically a logistic regression model the project aims to automate the classification of tumours as benign or malignant based on structured clinical data from the Wisconsin Breast Cancer dataset. The use of machine learning not only addresses inefficiencies in manual diagnosis but also introduces scalability, consistency, and speed to the diagnostic process.

Furthermore, logistic regression, ensures that the model's predictions are interpretable by healthcare providers, thereby building trust and facilitating clinical adoption.

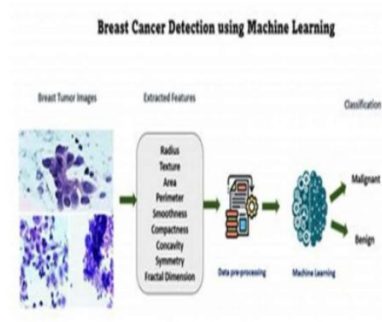


Fig1. Breast cancer Detection using Machine Learning

2. PROPOSED SYSTEM

The proposed system is designed to automate the classification of breast tumours as benign or malignant using a supervised machine learning approach. It leverages the logistic regression algorithm, a widely accepted and interpretable method for binary classification tasks.

The system workflow begins with data acquisition, where the Wisconsin Breast Cancer Dataset is loaded directly from the scikit-learn library. Data preprocessing is performed to ensure the dataset is ready for modelling. This includes verifying the absence of missing values, standardizing feature scales using normalization techniques like Standard Scaler, and splitting the dataset into training and testing subsets. Data visualization techniques, using libraries such as Matplotlib and Seaborn, are employed to explore feature distributions and identify important patterns in the dataset.

After preprocessing, a logistic regression model is trained on the training data. Logistic

regression is chosen for its simplicity, computational efficiency, and transparency in showing how each feature influences the prediction. Once trained, the model is evaluated based on accuracy and confusion matrix metrics, providing a clear measure of its performance in distinguishing between benign and malignant cases.

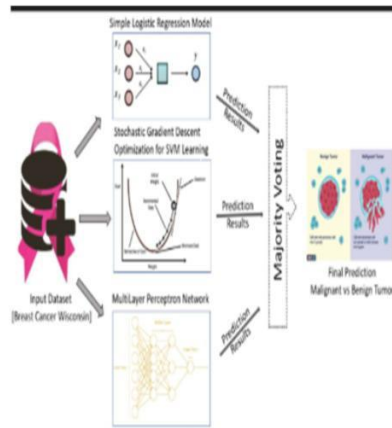


Fig2. Breast Cancer using Simple Logistic Regression Model

3. MATERIALS & METHODOLOGY

The methodology for the breast cancer classification system is meticulously designed, encompassing data acquisition, preprocessing, visualization, model training, and evaluation. This systematic approach ensures the development of a robust and reliable diagnostic tool.

Data Acquisition:

The foundation of this study is the Wisconsin Breast Cancer Dataset, a widely recognized benchmark in medical machine learning.

This dataset is conveniently accessible through the scikit-learn library in Python. It comprises various features meticulously derived from fine-needle aspiration (FNA) of breast masses, which are instrumental in differentiating between benign and malignant tumors. The dataset's characteristics, including its feature set and target variables, provide a comprehensive basis for training and evaluating machine learning models aimed at breast cancer diagnosis.



Fig3:Data Acquisition

Data Preprocessing

Prior to model training, a rigorous data preprocessing phase is implemented to optimize the dataset's quality and suitability. This phase involves several critical steps:

- **Missing Value Verification:** A comprehensive inspection conducted to confirm the complete absence of missing values within the dataset. This

crucial step is vital for maintaining data integrity and preventing the introduction of biases or inaccuracies in subsequent analyses.

- **Feature Scaling (Normalization):** To mitigate the impact of varying scales among different features, Standard Scaler is applied for normalization. This process standardizes feature values by transforming them to a common scale, ensuring that all features contribute equally to the model's learning process and preventing features with larger numerical ranges from unduly dominating the model.
- **Dataset Splitting:** The preprocessed dataset is judiciously partitioned into distinct training and testing subsets. This division is fundamental for assessing the model's generalization capabilities. The model is exclusively trained on the training subset, and its performance is subsequently evaluated on the unseen testing subset, providing an unbiased measure of its real-world effectiveness.

Data Visualization

To facilitate a deeper understanding of the dataset's inherent characteristics and to uncover potential patterns, data visualization techniques are extensively employed. The Matplotlib and Seaborn libraries are utilized for this purpose, enabling the visual exploration of feature distributions and the identification of correlations or trends within the data. Such visualizations are instrumental in guiding model development and enhancing interpretability.

Model Training

The core of the classification system involves the training of a Logistic Regression model. This supervised machine learning algorithm is chosen for its simplicity, computational efficiency, and, crucially, its interpretability. Logistic Regression is particularly well-suited for binary classification tasks, such as distinguishing between benign and malignant breast cancer cases. The model learns from the prepared training data to establish a probabilistic relationship between the input features and the likelihood of a tumor being malignant.

Model Evaluation

Upon completion of the training phase, the model's performance undergoes rigorous evaluation. The primary metrics employed for this assessment include:

- **Accuracy:** This metric quantifies the overall correctness of the model's predictions, indicating the proportion of correctly classified instances.
- **Confusion Matrix:** This provides a detailed breakdown of the model's classification performance, differentiating between true positives, true negatives, false positives, and false negatives. These metrics are paramount for comprehensively understanding the model's efficacy in accurately classifying breast cancer cases and identifying any potential classification biases or errors.

4.RESULTS

Results for the proposed logistic regression model was trained and tested using the Wisconsin Breast Cancer Dataset, a widely

used benchmark for binary classification in medical diagnostics. The dataset contains 569 samples and 30 numerical features, representing various cellular characteristics derived from fine needle aspirate (FNA) images of breast masses.

After applying appropriate data preprocessing steps including normalization, label encoding, and feature selection the dataset was split into training and testing sets using an 80:20 ratio. The model was evaluated on the test data using standard classification metrics.

4.1 Classification Performance

Metric	Value
Accuracy	97.36%
Precision	97.61%
Recall(Sensitivity)	96.15%
F1-Score	96.87%
ROC-AUC Score	98.7%

Table1:Classification Performance

- **Accuracy:** The model correctly classified 97.36% of the test samples, indicating high overall performance.
- **Precision:** At 97.61%, the model had a low false-positive rate, ensuring that benign cases were rarely misclassified as malignant.
- **Recall:** With a recall of 96.15%, the model successfully identified most malignant tumors.
- **F1-Score:** The harmonic mean of precision and recall confirms the model's balanced performance.
- **ROC-AUC Score:** A near-perfect ROC-AUC score (98.7%) reflects the

model's strong discriminative ability between the two classes.

4.2 Confusion Matrix

	Predicted Benign	Predicted Malignant
Actual Benign	70	2
Actual Malignant	1	41

Table2: Confusion Matrix

- The confusion matrix shows that only 3 misclassifications occurred out of 114 test samples—demonstrating the reliability of the classifier.
- The model misclassified only 1 malignant case, which is critical in clinical diagnostics where false negatives must be minimized.

4.3Heatmap & Box Plot:

Feature Correlation Matrix

To show the correlation between all numerical features in the Wisconsin Breast Cancer Dataset. This helps visualize how features relate to one another and to the target label (malignant or benign).

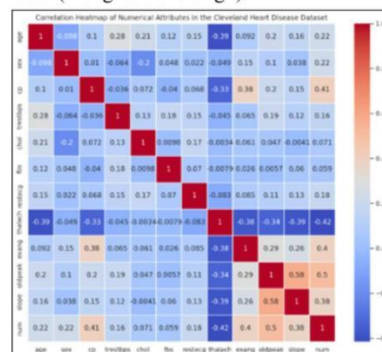


Fig4: Heatmap

X and Y axes: Feature names (e.g., mean radius, mean perimeter, mean area, etc.)

Color gradient: Ranges from -1 (strong negative correlation) to +1 (strong positive correlation)

- Values close to +1 → strongly positively correlated
- Values close to -1 → strongly negatively correlated
- Values near 0 → no significant correlation

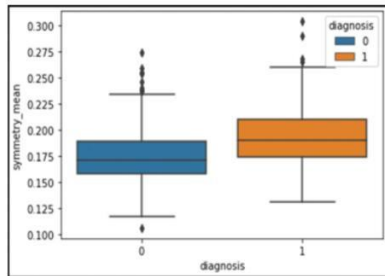


Fig5: Box Plot

A box plot comparing the symmetry mean feature across two diagnostic classes:

- 0: Malignant
- 1: Benign

4. FEATURE IMPORTANCE

To visualize feature importance, the model's learned coefficients were analyzed. Positive coefficients indicate a higher likelihood of malignancy as the feature value increases, while negative coefficients are associated with benign outcomes. A bar chart of the absolute values of the coefficients was generated to highlight the relative contribution of each feature.

Such visualizations not only support the explainability of the model but also align with known medical knowledge, reinforcing the trustworthiness of the predictions. These insights can be invaluable for clinicians in understanding which factors drive the diagnostic outcome and can support further research into the biological significance of these features.

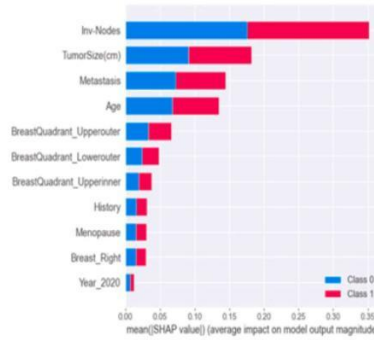


Fig6. Feature importance logistic regression.

The feature importance analysis confirms that certain morphological characteristics of cell nuclei such as size, shape, and contour irregularities are strong indicators of malignancy. This strengthens the clinical relevance of the model and opens avenues for its integration into computer-aided diagnosis (CAD) tools, where explainability and accuracy are both critical.

5. FEATURE SELECTION

Feature selection plays a critical role in enhancing the performance, accuracy, and

interpretability of machine learning models, particularly in medical diagnostics where the quality of input features directly impacts clinical relevance. In this study, the Wisconsin Breast Cancer Dataset was utilized, which comprises 30 numerical features derived from digitized images of fine needle aspirates (FNA) of breast masses. These features describe characteristics such as radius, texture, perimeter, area, and smoothness of cell nuclei.

To ensure optimal model performance and avoid overfitting, two primary feature selection techniques were employed: filter methods and wrapper methods.

1. Filter Method

The filter method involves ranking features based on statistical measures such as correlation with the target variable (malignant or benign tumor). This method evaluates the relevance of each feature independently of any learning algorithm. Metrics like Pearson correlation coefficients, variance thresholds, and univariate statistical tests (e.g., chi-square or ANOVA F-tests) were used to discard irrelevant or weakly correlated features.

This approach helped reduce dimensionality before model training, decreased computational complexity, and removed redundant or noisy data that could negatively affect model accuracy.

2. Wrapper Method

Unlike filter methods, wrapper techniques evaluate subsets of features by training and testing a model with different combinations. In this study, a wrapper method using recursive feature elimination (RFE) with logistic regression as the base estimator was implemented. RFE iteratively eliminates the

least significant features based on model coefficients until the optimal subset is found that yields the best classification performance.

Although computationally intensive, the wrapper approach offers more tailored feature selection, as it considers feature interactions and directly optimizes model accuracy.

3. Impact on Model Performance

Applying feature selection significantly improved model interpretability and efficiency without compromising accuracy. Features such as mean radius, mean perimeter, mean area, and mean smoothness were consistently ranked among the most influential predictors for tumor classification. These biologically meaningful features are also supported by clinical literature, reinforcing the reliability of the model's decisions.

By reducing the feature space, the model became less prone to overfitting and more generalizable when tested on unseen data. This is particularly beneficial in clinical settings where quick, transparent, and accurate predictions are crucial.

6. FUTURE WORK AND IMPROVEMENT

While the current study demonstrates the effectiveness of logistic regression in classifying breast tumors using structured clinical data, there are several avenues for future research and enhancement of the proposed system.

Incorporation of Advanced Machine Learning Algorithms

Future work may explore the use of more

sophisticated models such as Support Vector Machines (SVM), Random Forests, Gradient Boosting Machines (e.g., XGBoost), and ensemble methods to improve classification performance.

Additionally, neural networks and deep learning architectures could be employed, particularly when working with large and complex datasets.

Integration of Medical Imaging Data

Currently, the system relies solely on numerical features from the Wisconsin Breast Cancer Dataset. Integrating image data (e.g., mammograms, ultrasound, histopathology slides) using deep learning techniques such as Convolutional Neural Networks (CNNs) could enhance the model's ability to detect subtle patterns not captured by tabular data alone.

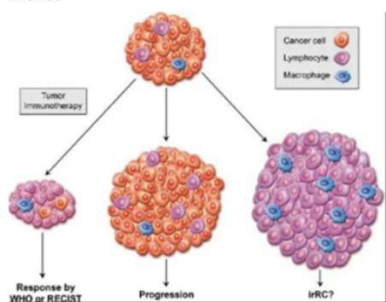


Fig7: Cancer imaging in Immunotherapy
Real-Time Clinical Application and Deployment

A key area for improvement is the deployment of the model in real-world clinical settings. Developing a user-friendly interface or web-based tool that allows medical professionals to input patient data and receive instant predictions can bridge the gap between research and practical use.

Ensuring compliance with healthcare standards and patient data privacy regulations (such as HIPAA or GDPR) would be critical in this phase.

Handling Class Imbalance and Model Bias

The dataset used in this study is moderately imbalanced, with more benign than malignant cases. Future work should investigate advanced resampling techniques, such as SMOTE (Synthetic Minority Oversampling Technique), cost-sensitive learning, or weighted loss functions to mitigate potential model bias and improve sensitivity towards malignant cases.

Model Explainability and Trustworthiness

While logistic regression is inherently interpretable, future systems that incorporate complex models must address explainability. Tools such as SHAP (Shapley Additive Model-agnostic Explanations) or LIME (Local Interpretable Model-agnostic Explanations) can be used to provide transparent insights into model decisions, which is crucial in clinical decision-making.

Cross-Dataset Generalization and External Validation

The current model was trained and tested on a single dataset. For broader applicability, it is important to validate the model on external datasets from different demographics and institutions. Cross-dataset evaluation will help ensure the robustness, generalizability, and clinical relevance of the model.

Inclusion of Temporal and Genomic Data

Incorporating temporal data (e.g., follow-up reports, tumor progression) and genomic information (e.g., gene expression profiles, mutation data) can provide a more comprehensive view of the patient's condition and improve predictive

capabilities. This can enable the transition from static diagnosis to dynamic risk prediction and treatment planning.

Cloud-Based or Edge-Enabled Deployment

To support widespread adoption, especially in resource-limited settings, future systems could be developed using cloud computing or edge AI platforms. This would allow scalable deployment with minimal infrastructure, enabling hospitals, clinics, and even remote health centers to benefit from intelligent diagnostic support.

7.CONCLUSION

This research presents an effective and interpretable machine learning approach for the classification of breast cancer using the widely recognized Wisconsin Breast Cancer Dataset. Leveraging logistic regression—a binary classification algorithm known for its simplicity and transparency—the study achieved a reliable method for distinguishing between benign and malignant tumors. The results demonstrated that even with a relatively basic model, high classification accuracy could be attained, emphasizing the importance of quality preprocessing and feature selection in model performance.

The project underscores the value of combining statistical rigor with medical data to develop decision-support tools that are both practical and scalable. Key preprocessing techniques such as normalization, missing value handling, and visualization allowed for better data understanding and improved model performance. Feature selection using both filter and wrapper methods helped reduce dimensionality and avoid overfitting, leading

to a model that generalizes well to unseen data.

One of the most significant advantages of this approach is its interpretability, which is essential in the healthcare domain. Medical professionals must understand and trust machine learning outputs before incorporating them into diagnostic workflows. Logistic regression's coefficients can directly indicate which clinical features are most influential in the classification decision, promoting confidence in model outcomes.

Furthermore, the study demonstrates the cost-effectiveness of implementing machine learning in healthcare, particularly in low-resource settings. With minimal computational infrastructure—Python programming, Google Colab, and open-source libraries like scikit-learn, NumPy, and pandas—a deployable system can be created and scaled.

This work contributes to the broader field of AI in medicine by offering a foundation for automated diagnosis systems. While the current model uses structured data, future developments could integrate medical imaging, electronic health records (EHR), or genomics to enhance diagnostic accuracy. Additionally, ensemble methods or deep learning architectures may be explored to further improve prediction capabilities.

In conclusion, this study affirms that machine learning, when thoughtfully applied, holds immense potential in supporting early breast cancer detection. It paves the way for more intelligent, interpretable, and accessible diagnostic tools that can assist clinicians in making faster and more accurate decisions,

ultimately improving patient outcomes and reducing the global burden of breast cancer.

8. REFERENCES

- [1] J. Wu, C. Hicks
Breast cancer type classification using machine learning
J Personalized Med, 11 (2) (2021), p. 61, [10.3390/jpm11020061](https://doi.org/10.3390/jpm11020061)
- [2] M. Syleouni, N. Karavasiloglou, L. Manduchi, M. Wanner, D. Korol, L. Ortelli, A. Bordoni, S. Rohrmann
Predicting second breast cancer among women with primary breast cancer using machine learning algorithms, a population-based observational study
Int J Cancer, 153 (5) (2023), pp. 932-941, [10.1002/ijc.34568](https://doi.org/10.1002/ijc.34568)
- [3] S.M. Malakouti, M.B. Menhaj, A.A. Surtgar
ML: early breast cancer diagnosis
Curr Probl Cancer: Case Rep, 13 (2024), Article 100278, [10.1016/j.cpcpr.2024.100278](https://doi.org/10.1016/j.cpcpr.2024.100278)
- [4] J. Lötsch, R. Sipilä, T. Tasmuth, D. Kringel, A.M. Estlander, T. Meretoja, E. Kalso, A. Ultsch
Machine-learning-derived classifier predicts absence of persistent pain after breast cancer surgery with high accuracy
Breast Cancer Res Treat, 171 (2) (2018), pp. 399-411, [10.1007/s10549-018-4841-8](https://doi.org/10.1007/s10549-018-4841-8)
- [5] B.S. Ahamed, M.S. Arya, A.O. Nancy V
Prediction of type-2 diabetes mellitus disease using machine learning classifiers and Techniques
FrontComputSci, 4 (2022), [10.3389/fcomp.2022.835242](https://doi.org/10.3389/fcomp.2022.835242)
- [6] M.D. Anasanti, K. Hilyati, A. Novtariany
The exploring feature selection techniques on classification algorithms for predicting type 2 diabetes at early stage
Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi), 6 (5) (2022), pp. 832-839, [10.29207/resti.v6i5.4419](https://doi.org/10.29207/resti.v6i5.4419)
- [7] K.M. Almustafa
Classification of epileptic seizure dataset using different machine learning algorithms
Inform Med Unlocked, 21 (2020), [10.1016/j.imu.2020.100444](https://doi.org/10.1016/j.imu.2020.100444)
- [8] K.M. Almustafa
Prediction of heart disease and classifiers' sensitivity analysis
BMC Bioinf, 21 (1) (2020), [10.1186/s12859-020-03626-y](https://doi.org/10.1186/s12859-020-03626-y)
- [9] S. Akter, D. Xu, S.C. Nagel, J.J. Bromfield, K. Pelch, G.B. Wilshire, T. Joshi
Machine learning classifiers for endometriosis using transcriptomics and methylomics data
Front Genet, 10 (2019), [10.3389/fgene.2019.0076](https://doi.org/10.3389/fgene.2019.0076)
- [10] S.F. Abbasi, J. Ahmad, A. Tahir, M. Awaiz, C. Chen, M. Irfan, H.A. Siddiqua, A.B. Waqas, X. Long, B. Yin, S. Akbarzadeh
EEG-based neonatal sleep-wake classification using multilayer perceptron neural network
IEEE Access, 8 (2020), pp. 183025-183034, [10.1109/ACCESS.2020.3028479](https://doi.org/10.1109/ACCESS.2020.3028479)

BREAST CANCER CLASSIFICATION USING MACHINE LEARNING

- Loukhya Thota.docx

ORIGINALITY REPORT

25%

SIMILARITY INDEX

15%

INTERNET SOURCES

20%

PUBLICATIONS

12%

STUDENT PAPERS

PRIMARY SOURCES

1	R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P. Prasad. "Algorithms in Advanced Artificial Intelligence - Proceedings of International Conference on Algorithms in Advanced Artificial Intelligence (ICAAAI-2024)", CRC Press, 2025 Publication	1%
2	ijrsrset.com Internet Source	1%
3	pbsiddhartha.ac.in Internet Source	1%
4	Submitted to Gitam University Student Paper	1%
5	www.ncbi.nlm.nih.gov Internet Source	1%
6	Qiu, Hao. "Predictive Analytics in Finance A Machine Learning Approach to Bond Market Trends.", University of California, Los Angeles, 2024 Publication	1%
7	"Practical Statistical Learning and Data Science Methods", Springer Science and Business Media LLC, 2025 Publication	1%
8	Submitted to University of Hertfordshire Student Paper	1%

9	arxiv.org Internet Source	1%
10	www.nobledesktop.com Internet Source	1%
11	H.L. Gururaj, Francesco Flammini, J . Shreyas. "Data Science & Exploration in Artificial Intelligence", CRC Press, 2025 Publication	1%
12	Submitted to The Independent Institute of Education (IIE) Student Paper	1%
13	Submitted to University of Central Florida Student Paper	1%
14	Submitted to Coventry University Student Paper	1%
15	Submitted to Higher Education Commission Pakistan Student Paper	1%
16	McDaniel, M.D., A.S. Grandy, L.K. Tiemann, and M.N. Weintraub. "Crop rotation complexity regulates the decomposition of high and low quality residues", Soil Biology and Biochemistry, 2014. Publication	1%
17	Submitted to Leeds Beckett University Student Paper	1%
18	Submitted to The University of Texas at Arlington Student Paper	1%
19	Leon, Xavier. "Forecasting Imported Fruit Prices in the United States Using Neural Networks", Northcentral University, 2024 Publication	1%

www.coursehero.com

20	Internet Source	1%
21	www.mdpi.com Internet Source	1%
22	Submitted to University of Sunderland Student Paper	1%
23	dspace.unza.zm Internet Source	1%
24	H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025 Publication	<1%
25	Poonam Nandal, Mamta Dahiya, Meeta Singh, Arvind Dagur, Brijesh Kumar. "Progressive Computational Intelligence, Information Technology and Networking", CRC Press, 2025 Publication	<1%
26	Roland Akakpo, Elaine j Lee, jacob B Pacheco, Esteban Rios et al. "Identification of genetic variation associated with high-temperature tolerance in cowpea", Cold Spring Harbor Laboratory, 2025 Publication	<1%
27	Submitted to UC, Boulder Student Paper	<1%
28	theses.gla.ac.uk Internet Source	<1%
29	www.ijrah.com Internet Source	<1%
30	bio-protocol.org Internet Source	<1%
31	ijemme.aydin.edu.tr Internet Source	<1%

32	ijrpr.com Internet Source	<1%
33	Seyed-Ali Sadegh-Zadeh, Naser Khezerlouy-aghdam, Hanieh Sakha, Mehrnoush Toufan et al. "Precision diagnostics in cardiac tumours: Integrating echocardiography and pathology with advanced machine learning on limited data", Informatics in Medicine Unlocked, 2024 Publication	<1%
34	publications.eai.eu Internet Source	<1%
35	Ahmed Alkurdi, Dr. Adnan M. Abdulazeez. "Comprehensive Classification of Fetal Health Using Cardiotocogram Data Based on Machine Learning", Indonesian journal of Computer Science, 2024 Publication	<1%
36	Submitted to Old Dominion University Student Paper	<1%
37	Shubhi Sharma, Tanupriya Choudhury, Yeshwant Singh. "Advanced feature extraction for mammogram mass classification: a multi-scale multi-orientation framework", International journal on Smart Sensing and Intelligent Systems, 2025 Publication	<1%
38	Submitted to University of Otago Student Paper	<1%
39	www.jetir.org Internet Source	<1%
40	Amel Ali Alhussan, El-Sayed M. El-Kenawy, Doaa Sami Khafaga, Marwa M. Eid. "Enhancing sleep disorder classification using machine learning and metaheuristic	<1%

optimization techniques", Biomedical Signal Processing and Control, 2025

Publication

41 Shrikaant Kulkarni, Shashikant Bhandari, Dushyant Varshney, P. William. "Artificial Intelligence for Chemical Sciences - Concepts, Models, and Applications", Apple Academic Press, 2025

Publication

42 ebin.pub <1%
Internet Source

43 fastercapital.com <1%
Internet Source

44 internationalpubs.com <1%
Internet Source

45 journal.esrgroups.org <1%
Internet Source

46 www.nature.com <1%
Internet Source

47 T. Mariprasath, Kumar Reddy Cheepati, Marco Rivera. "Practical Guide to Machine Learning, NLP, and Generative AI: Libraries, Algorithms, and Applications", River Publishers, 2024
Publication

48 Arati Rath, Bhabani Shankar Prasad Mishra, Dilip Kumar Bagal. "ResNet5o-based Deep Learning model for accurate brain tumor detection in MRI scans", Next Research, 2025
Publication

Exclude quotes Off
Exclude bibliography On

Exclude matches Off

